2.1. 머신 수준 ISA 버전 1.13

한국어 비공식 번역 · RISC-V ISA 명세 20260120

원문 machine.html

원문의 절 번호와 표 번호를 유지했다. 명령어 및 레지스터 이름은 원문 표기를 사용하며, 비트 도식은 동일한 필드와 비트 범위를 표로 재현했다.

이 장은 RISC-V 하트에서 가장 높은 특권 모드인 머신 모드(M모드)에서 사용할 수 있는 머신 수준 동작을 설명한다. M모드는 하드웨어 플랫폼에 저수준으로 접근하는 데 사용되며, 리셋 시 처음 진입하는 모드이다. 하드웨어로 직접 구현하기가 너무 어렵거나 비용이 큰 기능을 구현하는 데에도 M모드를 사용할 수 있다.

RISC-V 머신 수준 ISA에는 공통 핵심 부분이 있으며, 지원하는 다른 특권 수준과 하드웨어 구현의 세부 사항에 따라 확장된다.

2.1.1. 머신 수준 CSR

M모드 코드는 이 절의 머신 수준 CSR 외에도 더 낮은 특권 수준의 모든 CSR에 접근할 수 있다.

2.1.1.1. 머신 ISA 레지스터 misa

`misa` CSR은 하트가 지원하는 ISA를 보고하는 WARL 읽기/쓰기 레지스터이다. 모든 구현체에서 이 레지스터를 읽을 수 있어야 한다. 다만 값 0을 반환하여 `misa`가 구현되지 않았음을 나타낼 수 있으며, 이 경우 CPU 기능은 별도의 비표준 메커니즘으로 확인해야 한다.

그림 1. 머신 ISA 레지스터 misa

비트 범위	필드
MXLEN-1:MXLEN-2	MXL[1:0]
MXLEN-3:26	0 WARL
25:0	Extensions[25:0] WARL

MXL(Machine XLEN) 필드는 표 1과 같이 기본 정수 ISA의 본래 폭을 인코딩한다. MXL은 읽기 전용이다. `misa`가 0이 아니면 MXL은 M모드의 유효 XLEN을 나타내며, 이 상수를 MXLEN이라고 한다. XLEN은 MXLEN보다 클 수 없지만 낮은 특권 모드에서는 MXLEN보다 작을 수 있다.

표 1. misa의 MXL 필드 인코딩

MXL	XLEN
1	32
2	64
3	예약됨

`misa` CSR의 폭은 MXLEN비트이다.

> **주석** 반환된 `misa` 값의 부호를 검사하는 분기로 기본 폭을 빠르게 알아낼 수 있다. 필요하면 한 비트 왼쪽 시프트한 뒤 다시 부호를 검사하는 분기를 사용한다. 이러한 검사는 하트의 레지스터 폭 MXLEN을 몰라도 어셈블리 코드로 작성할 수 있다. 기본 폭은 MXLEN = 2^{MXL+4}로 주어진다. `misa`가 0이어도 레지스터에 즉시값 2를 넣고 31비트 왼쪽 시프트하여 기본 폭을 알아낼 수 있다. 결과가 0이면 RV32이고, 아니면 RV64이다.

Extensions 필드는 알파벳 문자마다 한 비트씩 사용하여 표준 확장의 존재를 인코딩한다. 비트 0은 A 확장, 비트 1은 B 확장, ..., 비트 25는 Z 확장을 나타낸다. RV32I와 RV64I 기본 ISA에서는 I비트가, RV32E와 RV64E에서는 E비트가 설정된다. Extensions는 WARL 필드이며, 지원 ISA를 변경할 수 있는 구현체에서는 쓰기 가능한 비트를 포함할 수 있다.

리셋 시 Extensions 필드는 지원되는 확장의 최대 집합을 포함해야 하며, I와 E를 모두 사용할 수 있으면 E보다 I를 선택해야 한다.

`misa`의 대응 비트를 지워 표준 확장을 비활성화하면, 그 확장이 정의하거나 수정하는 명령어 및 CSR은 해당 확장이 구현되지 않은 것처럼 원래 정의된 동작 또는 예약된 동작으로 돌아간다.

> **주석** 특정 RISC-V 실행 환경에서 명령어, 확장 또는 기타 ISA 기능이 구현되었는지는 일반적으로 그 환경에서 관찰 가능한 실행 동작을 기준으로 판단한다. 예를 들어 비특권 ISA가 F 확장에 대해 정의한 명령어들이 명세대로 실행되는 경우에만, 해당 실행 환경에 F 확장이 구현되었다고 한다.
>
> 이 정의에 따르면 `misa` 비트를 지워 확장을 비활성화하면 M모드에서는 그 확장이 구현되지 않은 것으로 간주된다. 예를 들어 `misa.F=0`이면 F 확장 명령어가 비특권 ISA의 요구대로 동작하지 않고 불법 명령어 예외를 발생시킬 수 있으므로, M 모드에 F 확장이 구현되지 않은 상태가 된다.
>
> 구현 여부를 관찰 가능한 동작만으로 정의하는 것은 이 단어의 일반적 사용법과 다를 수 있다. 일반적으로는 '구현되었지만 비활성화됨'이라는 표현을 허용할 수 있으나, 이 문서에서는 모순이다. 비활성화되면 해당 기능이 구현된 것으로 인정받기 위해 필요한 동작을 하지 않기 때문이다. 마찬가지로 '구현되고 활성화됨'은 중복 표현이며, '구현됨'만으로 충분하다.

향후 사용을 위해 예약된 모든 비트는 읽을 때 0을 반환해야 한다.

표 2. misa의 Extensions 필드 인코딩

비트	문자	설명
0	A	원자적 연산 확장
1	B	B 확장
2	C	압축 명령어 확장
3	D	배정밀도 부동소수점 확장
4	E	RV32E/64E 기본 ISA
5	F	단정밀도 부동소수점 확장
6	G	예약됨
7	H	하이퍼바이저 확장
8	I	RV32I/64I 기본 ISA
9	J	예약됨
10	K	예약됨
11	L	예약됨
12	M	정수 곱셈/나눗셈 확장
13	N	사용자 수준 인터럽트 확장용으로 잠정 예약
14	O	예약됨
15	P	Packed-SIMD 확장용으로 잠정 예약
16	Q	사배정밀도 부동소수점 확장

비트	문자	설명
17	R	예약됨
18	S	슈퍼바이저 모드 구현됨
19	T	예약됨
20	U	사용자 모드 구현됨
21	V	벡터 확장
22	W	예약됨
23	X	비표준 확장 존재
24	Y	예약됨
25	Z	예약됨

비표준 확장이 하나라도 있으면 X비트가 설정된다.

B비트가 1이면 구현체는 Zba, Zbb, Zbs 확장이 제공하는 명령어를 지원한다. B비트가 0이면 Zba, Zbb, Zbs 중 하나 이상을 지원하지 않을 수 있음을 나타낸다.

M비트가 1이면 M 확장이 정의하는 모든 곱셈 및 나눗셈 명령어를 지원한다. M비트가 0이면 해당 명령어를 지원하지 않을 수 있음을 나타낸다. 다만 Zmmul 확장을 지원하면 M비트 값과 관계없이 Zmmul이 지정한 곱셈 명령어를 지원한다.

S비트가 1이면 슈퍼바이저 모드를 지원한다. S비트가 0이면 슈퍼바이저 모드를 지원하지 않을 수 있다. U비트가 1이면 사용자 모드를 지원한다. U비트가 0이면 사용자 모드를 지원하지 않을 수 있다.

> **주석** `misa` CSR은 CPU 기능에 대한 기초적인 목록을 머신 모드 코드에 제공한다. 머신 모드에서는 다른 머신 레지스터를 탐색하고 부팅 과정에서 시스템의 다른 ROM 저장 공간을 조사하여 더 자세한 정보를 얻을 수 있다. 낮은 특권 수준에서는 CPU 레지스터를 직접 읽는 대신 환경 호출을 실행하여 해당 수준에서 사용할 수 있는 기능을 확인하도록 요구한다. 이를 통해 가상화 계층은 각 수준에서 관찰되는 ISA를 바꿀 수 있고, 하드웨어 설계 부담 없이 훨씬 풍부한 명령 인터페이스를 지원할 수 있다.

E비트는 읽기 전용이다. `misa` 전체가 읽기 전용 0인 경우를 제외하면 E비트는 항상 I비트의 반대 값으로 읽힌다. 실행 환경이 RV32E와 RV32I를 모두 지원하면 소프트웨어는 I비트를 지워 RV32E를 선택할 수 있다.

ISA 기능 x가 기능 y에 의존할 때, x를 활성화하고 y를 비활성화하려 하면 두 기능 모두 비활성화된다. 예를 들어 F=0, D=1로 설정하면 F와 D가 모두 지워진다. 마찬가지로 U=0, S=1로 설정하면 U와 S가 모두 지워진다.

구현체는 둘 이상의 `misa` 필드의 조합에 추가 제약을 부과할 수 있다. 이 경우 해당 필드들은 합쳐서 하나의 WARL 필드처럼 동작한다. 지원되지 않는 조합을 쓰려 하면 해당 비트들은 지원되는 어떤 조합으로 설정된다.

`misa` 쓰기는 C 확장 비활성화 등의 방법으로 IALIGN을 증가시킬 수 있다. `misa`에 쓰려는 명령어가 IALIGN을 증가시키고, 바로 다음 명령어의 주소가 IALIGN비트에 맞게 정렬되어 있지 않으면 그 쓰기는 억제되고 `misa`는 바뀌지 않는다.

소프트웨어가 이전에 비활성화된 확장을 활성화하면, 해당 확장에 별도 규정이 없는 한 그 확장에만 연결된 모든 상태는 `UNSPECIFIED`이다.

> **주석** `misa`의 비트 25~0 중 하나가 1이면 대응 기능이 구현되었음을 뜻하지만, 역은 반드시 성립하지 않는다. 비트가 0이라고 해서 그 기능이 구현되지 않았다고 단정할 수는 없다. 기능이 구현되지 않으면 대응 연산 코드와 CSR은 반드시 불법이 되는 것이 아니라 예약되기 때문이다.

2.1.1.2. 머신 공급업체 ID 레지스터 mvendorid

`mvendorid` CSR은 코어 제공자의 JEDEC 제조업체 ID를 제공하는 32비트 읽기 전용 레지스터이다. 모든 구현체에서 읽을 수 있어야 하지만, 필드가 구현되지 않았거나 비상업적 구현임을 나타내기 위해 0을 반환할 수 있다.

그림 2. 공급업체 ID 레지스터 mvendorid

비트 범위	필드
31:7	Bank
6:0	Offset

JEDEC 제조업체 ID는 일반적으로 1바이트 연속 코드 `0x7f`의 시퀀스 뒤에 `0x7f`가 아닌 1바이트 ID를 붙여 인코딩한다. 각 바이트의 최상위 비트에는 홀수 패리티 비트가 있다. `mvendorid`는 1바이트 연속 코드의 개수를 Bank 필드에 인코딩하고, 마지막 바이트에서 패리티 비트를 제외한 값을 Offset 필드에 인코딩한다.

예를 들어 `0x7f`가 12번 나온 뒤 `0x8a`가 오는 JEDEC 제조업체 ID는 `mvendorid` CSR에서 `0x60a`로 인코딩된다.

> **주석** JEDEC 용어의 뱅크 번호는 연속 코드 개수보다 1 크다. 따라서 `mvendorid.Bank`는 JEDEC 뱅크 번호보다 1 작은 값을 인코딩한다.
>
> 이전에는 RISC-V International이 공급업체 ID를 할당하도록 할 예정이었으나, 이는 제조업체 ID 표준을 관리하는 JEDEC의 역할과 중복된다. 원문 작성 시점에 JEDEC 제조업체 ID 등록에는 일회성 비용 500달러가 든다.

2.1.1.3. 머신 아키텍처 ID 레지스터 marchid

`marchid` CSR은 하트의 기본 마이크로아키텍처를 인코딩하는 MXLEN비트 읽기 전용 레지스터이다. 모든 구현체에서 읽을 수 있어야 하지만, 필드가 구현되지 않았음을 나타내기 위해 0을 반환할 수 있다. `mvendorid`와 `marchid`의 조합은 구현된 하트 마이크로아키텍처의 유형을 고유하게 식별하는 것이 바람직하다.

그림 3. 머신 아키텍처 ID 레지스터 marchid

비트 범위	필드
MXLEN-1:0	marchid

오픈 소스 프로젝트의 아키텍처 ID는 RISC-V International이 전역적으로 할당하며, 0이 아닌 값이면서 최상위 비트(MSB)가 0이다. 상업용 아키텍처 ID는 각 상업 공급업체가 독립적으로 할당하지만 MSB가 1이어야 하고, 나머지 MXLEN-1비트가 모두 0이어서는 안 된다.

> **주석** 아키텍처 ID는 특정 조직보다 개발의 중심이 되는 프로젝트의 마이크로아키텍처를 나타내도록 의도되었다. 오픈 소스 설계를 상업적으로 제작할 때는 원래 아키텍처 ID를 유지하는 것이 바람직하며, 라이선스가 이를 요구할 수도 있다. 이는 파편화와 도구 지원 비용을 줄이고 원 제작자 표시에도 도움이 된다.
>
> 오픈 소스 아키텍처 ID는 RISC-V International이 관리하며, 공개되어 동작하는 오픈 소스 프로젝트에만 할당하는 것이 바람직하다. 등록된 공급업체는 상업용 아키텍처 ID를 독립적으로 관리할 수 있지만, 비공개 및 오픈 소스 마이크로아키텍처를 함께 사용할 때 충돌하지 않도록 오픈 소스 ID와 겹치지 않는 값(MSB=1)을 사용해야 한다.

아래 Implementation 필드의 규칙으로 같은 아키텍처 설계의 분기를 조직별 분기 등을 포함해 구분할 수 있다. `misa` 레지스터도 설계의 서로 다른 변형을 구분하는 데 도움이 된다.

2.1.1.4. 머신 구현 ID 레지스터 mimpid

`mimpid` CSR은 프로세서 구현 버전을 고유하게 인코딩한다. 모든 구현체에서 읽을 수 있어야 하지만 필드 미구현을 나타내기 위해 0을 반환할 수 있다. Implementation 값은 주변 시스템이 아니라 RISC-V 프로세서 자체의 설계를 반영하는 것이 바람직하다.

그림 4. 머신 구현 ID 레지스터 mimpid

비트 범위	필드
MXLEN-1:0	mimpid

주석 이 필드의 형식은 아키텍처 소스 코드 제공자가 결정한다. 다만 표준 도구는 앞뒤의 0을 제외한 16진수 문자열로 출력하는 경우가 많다. 따라서 사람이 읽기 쉽도록 Implementation 값을 왼 쪽 정렬하고, 즉 최상위 니블부터 채우며, 하위 필드를 니블 경계에 맞출 수 있다.

2.1.1.5. 하트 ID 레지스터 mhartid

`mhartid` CSR은 코드를 실행하는 하드웨어 스레드의 정수 ID를 담는 MXLEN비트 읽기 전용 레지스터이다. 모든 구현체에서 읽을 수 있어야 한다. 다중 프로세서 시스템에서 하트 ID가 연속으로 매겨질 필요는 없지만, 한 하트의 ID는 반드시 0이어야 한다. 하트 ID는 실행 환경 안에서 고유해야 한다.

그림 5. 하트 ID 레지스터 mhartid

비트 범위	필드
MXLEN-1:0	mhartid

주석 리셋 시 등 일부 상황에서는 정확히 하나의 하트만 특정 코드를 실행하도록 보장해야 한다. 따라서 한 하트가 알려진 ID 0을 갖도록 요구한다. 효율성을 위해 시스템 구현자는 시스템에 사용되는 최대 하트 ID의 크기를 줄이는 것을 목표로 하는 것이 바람직하다.

2.1.1.6. 머신 상태 레지스터 mstatus 및 mstatush

`mstatus`는 MXLEN비트 읽기/쓰기 레지스터이며, RV32의 형식은 그림 6, RV64의 형식은 그림 7과 같다. 이 레지스터는 하트의 현재 동작 상태를 추적하고 제어한다. S수준 ISA의 `sstatus`는 `mstatus`의 제한된 뷰를 제공한다.

그림 6. RV32 머신 모드 상태 레지스터 mstatus

비트 범위	필드
31	SD
30:25	WPRI
24	SDT
23	SPELP
22	TSR
21	TW
20	TVM
19	MXR
18	SUM
17	MPRV
16:15	XS[1:0]
14:13	FS[1:0]
12:11	MPP[1:0]
10:9	VS[1:0]

비트 범위	필드
8	SPP
7	MPIE
6	UBE
5	SPIE
4	WPRI
3	MIE
2	WPRI
1	SIE
0	WPRI

그림 7. RV64 머신 모드 상태 레지스터 mstatus

비트 범위	필드
63	SD
62:43	WPRI
42	MDT
41	MPELP
40	WPRI
39	MPV
38	GVA
37	MBE
36	SBE
35:34	SXL[1:0]
33:32	UXL[1:0]
31:25	WPRI
24	SDT
23	SPELP
22	TSR
21	TW
20	TVM
19	MXR
18	SUM
17	MPRV
16:15	XS[1:0]
14:13	FS[1:0]
12:11	MPP[1:0]

비트 범위	필드
10:9	VS[1:0]
8	SPP
7	MPIE
6	UBE
5	SPIE
4	WPRI
3	MIE
2	WPRI
1	SIE
0	WPRI

RV32에만 존재하는 mstatush는 그림 8 형식의 32비트 읽기/쓰기 레지스터이다. mstatush의 비트 30:4에는 일반적으로 RV64 mstatus의 비트 62:36과 같은 필드가 들어 있다. SD, SXL, UXL 필드는 mstatush에 존재하지 않는다.

그림 8. RV32 추가 머신 모드 상태 레지스터 mstatush

비트 범위	필드
31:11	WPRI
10	MDT
9	MPELP
8	WPRI
7	MPV
6	GVA
5	MBE
4	SBE
3:0	WPRI

2.1.1.6.1. mstatus의 특권 및 전역 인터럽트 활성화 스택

M모드와 S모드에는 각각 전역 인터럽트 활성화 비트 MIE와 SIE가 제공된다. 이 비트들은 주로 현재 특권 모드의 인터럽트 처리기에 대한 원자성을 보장하는 데 사용된다.

주석 전역 xIE 비트는 mstatus의 하위 비트에 배치되어, CSR 명령어 하나로 원자적으로 설정하거나 지울 수 있다.

하트가 특권 모드 x에서 실행 중이면 xIE=1일 때 인터럽트가 전역적으로 활성화되고 xIE=0일 때 전역적으로 비활성화된다. 더 낮은 특권 모드 w<x의 인터럽트는 해당 모드의 전역 wIE 값과 관계없이 항상 전역적으로 비활성화된다. 더 높은 특권 모드 y>x의 인터럽트는 해당 모드의 전역 yIE 값과 관계없이 항상 전역적으로 활성화된다.

높은 특권 수준의 코드는 낮은 특권 모드로 제어를 넘기기 전에 개별 인터럽트 활성화 비트를 사용하여 일부 높은 특권 모드 인터럽트를 비활성화할 수 있다. 슈퍼바이저 모드를 구현하지 않으면 SIE와 SPIE는 읽기 전용 0이다.

주석 높은 특권 모드 y는 낮은 특권 모드로 제어를 넘기기 전에 자신의 모든 인터럽트를 비활성화할 수도 있다. 하지만 그러면 동기 트랩, 마스킹 불가능 인터럽트 또는 리셋만으로 하트의 제어를 되찾을 수 있으므로 일반적이지 않다.

중첩 트랩을 지원하기 위해, 인터럽트에 응답할 수 있는 각 특권 모드 x는 인터럽트 활성화 비트와 특권 모드에 대한 2단계 스택을 갖는다. xPIE는 트랩 전 인터럽트 활성화 비트 값을 저장하고, xPP는 이전 특권 모드를 저장한다. xPP는 x 이하의 특권 모드만 저장할 수 있으므로 MPP는 2비트이고 SPP는 1비트이다. 특권 모드 y에서 x로 트랩하면 xPIE←xIE, xIE←0, xPP←y가 된다.

주석 낮은 특권 모드의 동기 또는 비동기 트랩은 보통 더 높은 특권 모드에서 처리되며 진입 시 인터럽트가 비활성화된다. 높은 수준의 트랩 처리기는 트랩을 처리한 뒤 저장된 정보로 복귀하거나, 중단된 문맥으로 바로 복귀하지 않을 경우 인터럽트를 다시 활성화하기 전에 특권 스택을 저장한다. 따라서 각 스택에는 한 항목만 있으면 된다.

MRET와 SRET는 각각 M모드 및 S모드 트랩에서 복귀하는 명령어이다. xRET를 실행할 때 xPP에 y가 들어 있다고 하면, xIE←xPIE, 특권 모드←y, xPIE←1, xPP←지원하는 최저 특권 모드가 된다. 최저 모드는 U모드를 구현하면 U, 아니면 M이다. y≠M이면 xRET는 MPRV도 0으로 설정한다.

주석 xRET에서 xPP를 지원하는 최저 특권 모드로 설정하면 2단계 특권 모드 스택 관리의 소프트웨어 버그를 식별하는 데 도움이 된다.

주석 트랩 처리기는 트랩의 처리 및 복귀에 필요한 핵심 상태 정보를 보존하는 단계에서 인터럽트를 활성화하거나 예외를 일으키지 않도록 설계해야 한다. 이 중요한 단계에서 발생한 예외나 인터럽트는 핵심 상태를 덮어써 최초 트랩 복구에 필요한 데이터를 잃게 할 수 있다. 또한 트랩 처리 경로의 코드에서 예외가 발생하면 무한한 트랩 반복으로 이어질 수 있다. 이를 막으려면 처리기의 동작 과정에서 예외를 식별하고 안전하게 관리하도록 세심하게 설계해야 한다.

xPP는 특권 모드 x 및 구현된 하위 특권 모드만 저장할 수 있는 WARL 필드이다. 모드 x를 구현하지 않으면 xPP는 읽기 전용 0이어야 한다.

주석 M모드 소프트웨어는 MPP에 특정 모드를 쓴 뒤 다시 읽어 그 모드의 구현 여부를 알아낼 수 있다. U와 M만 제공하는 머신에서는 MPP의 00과 11을 표현하기 위해 실제 저장 비트 하나만 필요하다.

2.1.1.6.2. mstatus의 이중 트랩 제어

이중 트랩은 보통 트랩 처리의 민감한 단계, 즉 트랩 처리기가 재진입 불가능한 상태에서 예외나 인터럽트가 발생할 때 생긴다. 이러한 재진입 불가능 상태는 보통 트랩 처리 및 복귀에 필요한 상태를 아직 보존하지 않은 초기 단계에서 나타난다. 이때 트랩이 발생하면 핵심 상태가 덮어써져 최초 트랩 복구에 필요한 데이터를 잃을 수 있다. 이 심각한 오류를 유발한 트랩을 **예상치 못한 트랩(unexpected trap)**이라고 한다. 트랩 처리기는 이 단계에서 인터럽트를 활성화하거나 예외를 발생시키지 않도록 설계된다. 그러나 예측할 수 없이 발생하는 하드웨어 오류 예외는 이중 트랩 위험 때문에 처리기 구현을 특히 어렵게 한다.

M모드 트랩 비활성화 비트 MDT는 Smdbltrp 확장이 도입한 WARL 필드이다. 리셋 시 MDT는 1로 설정된다. 명시적 CSR 쓰기로 MDT를 1로 설정하면 MIE가 0으로 지워진다. RV64에서는 같은 쓰기가 MIE에 어떤 값을 쓰든 이 지우기가 수행된다. 명시적 CSR 쓰기로 MIE를 1로 설정하려면 MDT가 이미 0이어야 하며, RV64에서는 같은 쓰기로 MDT를 0으로 설정하는 경우도 허용된다. RV32에서는 MDT가 `mstatush`에, MIE가 `mstatus`에 있다.

M모드로 트랩할 때 MDT가 0이면 1로 설정하고 예상대로 트랩을 전달한다. MDT가 이미 1이면 예상치 못한 트랩이다. Smrnmi 확장을 구현하면 RNMI로 인한 트랩은 MDT 상태와 관계없이 예상치 못한 트랩으로 보지 않으며 MDT를 설정하지 않는다. 다만 M모드 실행 중 `mnstatus.NMIE=0`일 때 발생한 트랩은 예상치 못한 트랩이다.

예상치 못한 트랩은 다음과 같이 처리한다.

- Smrnmi를 구현하고 `mnstatus.NMIE=1`이면 RNMI 처리기로 트랩한다. 예상치 못한 트랩이 `mepc` 및 `mcause`에 기록했을 값을 각각 `mnepc` 및 `mncause`에 기록한다. `mnstatus`의 특권 모드 정보 필드는 M모드를 나타내도록 설정하고 NMIE는 0으로 설정한다.

주석 따라서 이중 트랩 시 RNMI 처리기는 원래 트랩이 `mtval`과 `mtval2`에 보고할 정보를 받지 못한다. 필요하면 `mnepc`가 가리키는 명령어를 디코딩하고 소스 레지스터 내용을 검사하여 이 정보를 얻을 수 있다.

- Smrnmi를 구현하지 않았거나, 구현했어도 `mnstatus.NMIE=0`이면 `pc`를 포함한 아키텍처 상태를 갱신하지 않고 심각한 오류 상태(critical-error state)에 진입한다. 이 상태에서는 실행을 중지하고 NMI를 포함한 모든 인터럽트를 비활성화하며 플랫폼에 `critical-error` 신호를 출력한다. 이 상태에서 성능 카운터와 타이머를 갱신하는지는 UNSPECIFIED이다.

주석 하트의 `critical-error` 신호에 플랫폼이 어떻게 대응하는지는 플랫폼마다 다르다. 해당 하트 재시작이나 전체 플랫폼 재시작 등이 가능하다.

M모드에서 실행한 MRET와 SRET는 MDT를 0으로 설정한다. 새 특권 모드가 U, VS 또는 VU이면 `sstatus.SDT`도 0으로 설정한다. VU이면 추가로 `vsstatus.SDT`도 0으로 설정한다.

Smrnmi 확장의 MNRET는 새 특권 모드가 M이 아니면 MDT를 0으로 설정한다. 새 모드가 U, VS 또는 VU이면 `sstatus.SDT`도 0으로, VU이면 추가로 `vsstatus.SDT`도 0으로 설정한다.

2.1.1.6.3. mstatus의 기본 ISA 제어

RV64 하트에서 SXL과 UXL은 각각 S모드 및 U모드의 XLEN을 제어하는 WARL 필드이다. 인코딩은 표 1의 `misa.MXL`과 같다. S모드와 U모드의 유효 XLEN을 각각 SXLEN과 UXLEN이라고 한다.

MXLEN=32이면 SXL과 UXL은 존재하지 않으며 SXLEN=UXLEN=32이다. MXLEN=64이고 S모드를 지원하지 않으면 SXL은 읽기 전용 0이다. S모드를 지원하면 현재 SXLEN을 인코딩하는 WARL 필드이며, 항상 SXLEN=MXLEN이 되도록 읽기 전용으로 구현할 수도 있다. MXLEN=64이고 U모드를 지원하지 않으면 UXL은 읽기 전용 0이다. U모드를 지원하면 현재 UXLEN을 인코딩하는 WARL 필드이며, 항상 UXLEN=MXLEN 또는 UXLEN=SXLEN이 되도록 읽기 전용으로 구현할 수도 있다.

S모드가 구현되면 UXL의 유효 값 집합에는 UXLEN>SXLEN이 되는 값이 포함되지 않는다.

어떤 모드의 XLEN을 지원되는 최대 XLEN보다 작게 설정하면, 모든 연산은 설정된 XLEN보다 높은 소스 레지스터 비트를 무시하고 결과를 부호 확장하여 목적지 레지스터의 최대 지원 폭 전체를 채워야 한다. 마찬가지로 XLEN보다 높은 pc 비트는 무시하며 pc에 쓸 때 최대 지원 XLEN까지 부호 확장한다.

주석 구현별로 동작이 달라지는 것을 피하기 위해, 실제 하드웨어 레지스터 전체를 정의된 값으로 채우도록 요구한다. 하드웨어 복잡도를 줄이기 위해 아키텍처는 하위 특권 모드의 XLEN이 바로 위 특권 모드의 XLEN 이하인지 검사하도록 요구하지 않는다. 실제로 이러한 설정은 거의 항상 소프트웨어 버그이겠지만, 그런 경우에도 머신 동작은 명확히 정의된다.

일부 HINT는 `c.addi x8, 0`처럼 목적지 레지스터에 현재 값을 다시 쓰는 정수 연산 명령어로 인코딩된다. XLEN<MXLEN이고 목적지의 XLEN 이상 상위 비트가 모두 비트 XLEN-1과 같지는 않은 상태에서 이러한 HINT를 실행하면, 해당 상위 비트가 유지되는지 비트 XLEN-1의 복사본으로 덮어써지는지는 구현 정의이다.

주석 이 정의는 일부 HINT의 레지스터 쓰기 복귀를 생략하면서 다른 HINT는 일반 정수 연산과 동일하게 실행할 수 있게 한다. 이 구현 선택은 현재 XLEN보다 큰 XLEN으로 실행되는 특권 모드에서만 관찰할 수 있고 현재 특권 모드에서는 보이지 않는다.

2.1.1.6.4. mstatus의 메모리 접근 특권

MPRV(Modify PRiVilege)는 명시적 메모리 접근이 실행되는 유효 특권 모드를 변경한다. MPRV=0이면 현재 특권 모드의 주소 변환 및 보호 메커니즘을 사용하여 정상적으로 접근한다. MPRV=1이면 현재 모드가 MPP인 것처럼 로드 및 스토어 주소를 변환·보호하고 엔디언을 적용한다. 명령어 주소 변환과 보호는 MPRV의 영향을 받지 않는다. U모드를 지원하지 않으면 MPRV는 읽기 전용 0이다. MRET 또는 SRET로 M보다 낮은 특권 모드로 전환하면 MPRV도 0이 된다.

MXR(Make eXecutable Readable)은 로드가 가상 메모리에 접근할 때의 권한 해석을 바꾼다. MXR=0이면 읽기 가능으로 표시된 페이지(Sv32 페이지 테이블 엔트리의 R=1)에서만 로드가 성공한다. MXR=1이면 읽기 가능 또는 실행 가능(R=1 또는 X=1)인 페이지에서 로드가 성공한다. 페이지 기반 가상 메모리가 적용되지 않으면 MXR은 효과가 없다. S모드를 지원하지 않으면 MXR은 읽기 전용 0이다.

주석 MPRV와 MXR은 비정렬 로드 및 스토어 등 없는 하드웨어 기능을 에뮬레이션하는 M모드 루틴의 효율을 높이기 위해 고안되었다. MPRV는 소프트웨어 주소 변환을 없애고, MXR은 실행 전용 페이지의 명령어 워드를 로드할 수 있게 한다. 현재 특권 모드와 MPP가 지정한 모드는 XLEN 설정이 다를 수 있다.

MPRV=1이면 로드 및 스토어 주소는 2.1.1.6.3절 규칙에 따라 현재 XLEN이 MPP 모드의 XLEN인 것처럼 취급된다.

SUM(permit Supervisor User Memory access)은 S모드 로드 및 스토어가 가상 메모리에 접근할 때의 권한을 바꾼다. SUM=0이면 U모드 접근 가능 페이지(Sv32 PTE의 U=1)에 대한 S모드 메모리 접근은 폴트를 발생시킨다. SUM=1이면 허용된다. 페이지 기반 가상 메모리가 적용되지 않으면 효과가 없다. SUM은 보통 S모드 밖에서는 무시되지만 MPRV=1이고 MPP=S이면 적용된다. S모드를 지원하지 않거나 `satp.MODE`가 읽기 전용 0이면 SUM은 읽기 전용 0이다.

MXR과 SUM은 페이지 테이블 엔트리에 인코딩된 권한 해석에만 영향을 준다. 특히 PMA나 PMP로 인한 접근 폴트 예외 발생 여부에는 영향을 주지 않는다.

2.1.1.6.5. mstatus 및 mstatush의 엔디언 제어

`mstatus`와 `mstatush`의 MBE, SBE, UBE는 명령어 인출을 제외한 메모리 접근의 엔디언을 제어하는 WARL 필드이다. 명령어 인출은 항상 리틀 엔디언이다.

MBE는 M모드에서 수행하는 명령어 인출 이외의 메모리 접근(`mstatus.MPRV=0` 가정)을 리틀 엔디언(MBE=0) 또는 빅 엔디언(MBE=1)으로 제어한다. S모드를 지원하지 않으면 SBE는 읽기 전용 0이다. 지원하면 S모드의 명시적 로드 및 스토어를 리틀 엔디언(SBE=0) 또는 빅 엔디언(SBE=1)으로 제어한다. U모드를 지원하지 않으면 UBE는 읽기 전용 0이다. 지원하면 U모드의 명시적 로드 및 스토어를 리틀 엔디언(UBE=0) 또는 빅 엔디언(UBE=1)으로 제어한다.

페이지 테이블 등 슈퍼바이저 수준 메모리 관리 자료구조에 대한 암묵적 접근의 엔디언은 항상 SBE가 제어한다. SBE 변경은 해당 자료구조의 해석을 바꾸므로, 변경 전후에 계속 사용하는 자료구조가 하나라도 있으면 M모드 소프트웨어는 변경 후 `rs1=x0`, `rs2=x0`인 SFENCE.VMA를 실행해야 한다.

주석 같은 메모리 관리 자료구조를 리틀 엔디언과 빅 엔디언 양쪽으로 해석하는 것은 인위적인 상황에서나 생긴다. 실제로 SBE는 보통 실행 세계 전환(world switch)에서만 바뀌며, 이때 기존 및 새 자료구조를 다른 엔디언으로 재해석하지 않는다. 따라서 세계 전환에 원래 필요한 것 외에 추가 SFENCE.VMA는 필요하지 않다.

S모드를 지원하는 구현체는 SBE를 MBE의 읽기 전용 복사본으로 만들 수 있다. U모드를 지원하는 구현체는 UBE를 MBE 또는 SBE의 읽기 전용 복사본으로 만들 수 있다.

주석 MBE, SBE, UBE가 모두 읽기 전용 0이면 리틀 엔디언 메모리 접근만 지원한다. MBE가 읽기 전용 1이고, S모드와 U모드를 지원할 때 SBE와 UBE도 각각 읽기 전용 1이면 명령어 인출을 제외한 빅 엔디언 접근만 지원한다.

제I권은 하트의 주소 공간을 연속 주소에 놓인 2^{XLEN}바이트의 순환 시퀀스로 정의한다. 주소와 바이트 위치의 대응은 고정되어 엔디언의 영향을 받지 않는다. 엔디언은 메모리 바이트와 하프워드·워드 등 여러 바이트 값 사이의 매핑 순서를 결정한다.

표준 RISC-V ABI는 엔디언 혼용을 지원하지 않고 리틀 엔디언 전용 또는 빅 엔디언 전용일 것으로 예상한다. 그래도 한 엔디언의 OS가 반대 엔디언의 사용자 프로그램을 실행할 수 있도록 엔디언 제어를 정의하였다. 소프트웨어가 필요에 따라 메모리 접근 엔디언을 바꾸는 비표준 사용 가능성도 고려하였다.

RISC-V 명령어는 인코딩을 현재 엔디언 설정과 분리하여 하드웨어와 소프트웨어에 이점을 주기 위해 항상 리틀 엔디언이다. 그렇지 않으면 실행 중 엔디언이 동적으로 바뀔 수 있어도 어셈블러나 디스어셈블러가 의도된 엔디언을 항상 알아야 한다.

명령어 엔디언을 고정하면 신중히 작성한 소프트웨어는 위치 독립 코드처럼 바이너리 수준에서도 엔디언에 독립적일 수 있다. 다만 머신 명령어를 인코딩하거나 디코딩하는 소프트웨어는 영향을 받는다. 빅 엔디언 모드에서는 명시적 로드·스토어와 명령어의 엔디언이 반대이므로, 예를 들어 로드 후 및 스토어 전 바이트 순서를 바꾸어야 한다.

2.1.1.6.6. mstatus의 가상화 지원

TVM(Trap Virtual Memory)은 슈퍼바이저 가상 메모리 관리 동작을 가로채기 위한 WARL 필드이다. TVM=1이면 S 모드에서 `satp` CSR을 읽거나 쓰거나, SFENCE.VMA 또는 SINVAL.VMA를 실행하려 할 때 불법 명령어 예외가 발생한다. TVM=0이면 S모드에서 이 동작들이 허용된다. S모드를 지원하지 않으면 TVM은 읽기 전용 0이다.

주석 TVM은 게스트 OS를 전통적인 U모드 가상화 대신 S모드에서 실행하게 하여 가상화 효율을 높인다. 대부분의 S모드 CSR 접근에 트랩이 필요하지 않게 된다. `satp` 접근과 SFENCE.VMA·SINVAL.VMA 명령어를 트랩하면 ㅅㅐ도 페이지 테이블을 필요할 때 채우는 데 필요한 진입점을 제공할 수 있다.

TW(Timeout Wait)는 WFI 명령어(2.1.3.3절)를 가로채기 위한 WARL 필드이다. TW=0이면 다른 이유로 금지되지 않는 한 M보다 낮은 특권 모드에서도 WFI를 실행할 수 있다. TW=1이면 낮은 특권 모드의 WFI가 구현체가 정한 유한 시간 안에 완료되지 않을 때 불법 명령어 예외를 발생시킨다. 구현체는 TW=1이면 M보다 낮은 모드의 WFI에서 항상 불법 명령어 예외가 나도록 할 수 있으며, 실행 당시 전역 비활성화 상태의 대기 인터럽트가 있어도 마찬가지이다. M보다 낮은 모드가 없으면 TW는 읽기 전용 0이다.

주석 WFI를 트랩하면 현재 게스트에서 시간을 낭비하며 유휴 상태를 유지하는 대신 다른 게스트 OS로 전환할 수 있다.

S모드가 구현된 경우 U모드에서 WFI를 실행하면 TW 값과 관계없이 불법 명령어 예외가 발생한다. 단, 구현체가 정한 유한 시간 안에 명령어가 완료되는 경우는 예외이다.

TSR(Trap SRET)은 슈퍼바이저 예외 복귀 명령어 SRET를 가로채기 위한 WARL 필드이다. TSR=1이면 S모드에서 SRET를 실행하려 할 때 불법 명령어 예외가 발생한다. TSR=0이면 허용된다. S모드를 지원하지 않으면 TSR은 읽기 전용 0이다.

주석 H 확장을 제공하지 않는 구현체에서 하이퍼바이저 확장을 에뮬레이션하려면 SRET를 트랩해야 한다.

2.1.1.6.7. mstatus의 확장 문맥 상태

상당한 규모의 확장을 지원하는 것은 RISC-V의 주요 목표 중 하나이다. 따라서 특권 모드 코드, 특히 슈퍼바이저 수준 OS를 바꾸지 않고 임의의 사용자 모드 상태 확장을 지원할 수 있도록 표준 인터페이스를 정의한다.

주석 현재까지 부동소수점 CSR과 데이터 레지스터 외의 추가 상태를 정의하는 표준 확장은 V뿐이다.

FS[1:0] 및 VS[1:0] WARL 필드와 XS[1:0] 읽기 전용 필드는 부동소수점 장치 및 그 밖의 사용자 모드 확장 상태를 설정·추적하여 문맥 저장과 복원 비용을 줄이는 데 사용된다. FS는 `f0~f31`과 `fcsr`, `frm`, `fflags`를 포함한 부동소수점 상태를 인코딩한다. VS는 `v0~v31`과 `vcsr`, `vxrm`, `vxsat`, `vstart`, `vl`, `vtype`, `vlenb`를 포함한 벡터 확장 상태를 인코딩한다. XS는 추가 사용자 모드 확장과 그 관련 상태를 인코딩한다. 문맥 교환 루틴은 이 필드들을 검사하여 상태 저장이나 복원이 필요한지 빠르게 판단할 수 있다. 실제로 저장·복원이 필요하면 보통 추가 명령어와 CSR로 이를 수행하고 최적화한다.

주석 대부분의 문맥 교환에서는 부동소수점 장치나 기타 확장 중 하나 또는 둘 다의 상태 저장·복원이 필요하지 않을 것으로 예상하므로 SD비트로 빠르게 검사하게 하였다.

FS, VS, XS는 표 3의 같은 상태 인코딩을 사용한다. 가능한 네 값은 Off, Initial, Clean, Dirty이다.

표 3. FS와 VS 및 XS 상태 필드의 인코딩

상태 값	FS 및 VS의 의미	XS의 의미
0	Off 꺼 짐	모두 꺼 짐

상태 값	FS 및 VS의 의미	XS의 의미
1	Initial 초기	Dirty나 Clean은 없고 일부 켜짐
2	Clean 저장본과 일치	Dirty는 없고 일부 Clean
3	Dirty 변경 가능 상태	일부 Dirty

F 확장을 구현하면 FS는 읽기 전용 0이어서는 안 된다. F 확장과 S모드를 모두 구현하지 않으면 FS는 읽기 전용 0이다. S모드는 구현하지만 F 확장은 구현하지 않으면 FS를 읽기 전용 0으로 만들 수 있다.

주석 S모드는 있지만 F 확장이 없는 구현체는 FS를 읽기 전용 0으로 만들 수 있으나 의무는 아니다. 일부는 M모드로의 비가시적 트랩을 통해 S모드와 U모드 모두의 F 확장을 에뮬레이션하기 위해 읽기 전용 0으로 만들지 않을 수 있다.

v 레지스터를 구현하면 VS는 읽기 전용 0이어서는 안 된다. v 레지스터와 S모드를 모두 구현하지 않으면 VS는 읽기 전용 0이다. S모드는 구현하지만 v 레지스터는 구현하지 않으면 VS를 읽기 전용 0으로 만들 수 있다.

새 상태가 필요한 추가 사용자 확장이 없는 하트에서는 XS가 읽기 전용 0이다. 상태를 갖는 각 추가 확장은 XS 상태에 해당하는 값을 인코딩하는 CSR 필드를 제공한다. XS는 표 3과 같이 모든 확장 상태의 요약을 나타낸다.

주석 개별 확장의 인코딩이 XS와 다를 수는 있지만, XS는 사실상 모든 사용자 확장 상태 필드 중 최댓값을 보고한다.

SD는 FS, VS, XS 중 어느 하나가 확장 사용자 문맥을 메모리에 저장해야 하는 Dirty 상태를 나타내는지를 요약하는 읽기 전용 비트이다. FS, XS, VS가 모두 읽기 전용 0이면 SD도 항상 0이다.

확장 상태가 Off이면 해당 상태를 읽거나 쓰려는 명령어는 불법 명령어 예외를 발생시킨다. Initial이면 해당 상태는 일정한 초기 값을 갖는 것이 바람직하다. Clean이면 초기 값과 다를 수 있으나 마지막 문맥 교환 때 저장한 값과 일치한다. Dirty이면 마지막 문맥 저장 이후 수정되었을 가능성이 있다.

문맥을 저장할 때 특권 코드는 Dirty인 상태만 메모리에 쓰면 되고, 이후 상태를 Clean으로 되돌릴 수 있다. 문맥 복원 시에는 Clean일 때만 메모리에서 불러오면 되며, 복원 시 Dirty이어서는 안 된다. Initial이면 보안 취약점을 피하도록 일정한 초기 값으로 설정해야 하지만 메모리에 접근할 필요는 없다. 예를 들어 부동소수점 레지스터 전체를 즉시값 0으로 초기화할 수 있다.

특권 코드는 문맥 저장 전에 FS와 XS를 읽는다. 사용자 문맥을 재개할 때 FS는 특권 코드가 직접 설정하고, XS는 개별 확장 상태 레지스터에 써서 간접적으로 설정한다. 상태 필드는 특권 모드와 관계없이 명령어 실행 중에도 갱신된다.

사용자 모드 ISA 확장은 종종 기본 정수 레지스터보다 훨씬 큰 추가 상태를 포함한다. 일부 응용만 사용하거나 한 응용의 짧은 구간에서만 필요할 수도 있다. 성능을 높이기 위해 확장은 사용자 소프트웨어가 장치를 초기 상태로 돌리거나 꺼 버리는 추가 명령어를 정의할 수 있다. 예를 들어 보조 프로세서를 사용 전에 구성하고 사용 후 '구성 해제'할 수 있으며, 구성 해제 상태는 문맥 저장 관점에서 Initial이다. 구성 해제와 다음 구성(Dirty로 설정) 사이에 같은 응용이 계속 실행된다면 구성 해제 시 실제 초기화는 불필요하다. 상태가 해당 사용자 프로세스에만 속하므로, Initial은 매 구성 해제 때가 아니라 문맥 복원 때 상태를 상수로 초기화하게 할 수 있다.

사용자 명령어가 장치를 비활성화해 Off로 만들면 다시 켜기 전에 사용하는 후속 명령어는 불법 명령어 예외를 발생시킨다. 장치를 켜는 사용자 명령어는 그사이 다른 문맥이 장치를 사용했을 수 있으므로 상태가 적절히 초기화되도록 해야 한다.

FS 설정 변경은 부동소수점 레지스터 내용에 영향을 주지 않는다. FS=Off도 상태를 파괴하지 않고 FS=Initial도 내용을 지우지 않는다. VS 변경도 벡터 레지스터 내용에 영향을 주지 않는다. 다만 다른 확장은 Off에서 상태를 보존하지 않을 수 있다.

구현체는 실제로 수정되지 않았어도 Dirty로 보고하는 부정확한 부동소수점 변경 추적을 선택할 수 있다. 일부 구현에서는 상태를 바꾸지 않는 명령어도 Initial이나 Clean에서 Dirty로 전이시킬 수 있다. 변경 여부를 전혀 추적하지 않는 구현에서는 유효 FS 상태가 Off와 Dirty뿐이며, Initial이나 Clean을 쓰려 하면 Dirty가 된다.

> **주석** 이 FS 정의는 잘못된 추측 실행 결과로 Dirty가 되는 것을 금지하지 않는다. 일부 플랫폼은 잠재적인 부채널을 차단하기 위해 추측적으로 FS에 쓰는 것을 금지할 수 있다.

명령어가 부동소수점 레지스터나 `fcsr`에 명시적 또는 암묵적으로 쓰되 내용을 바꾸지 않고, FS가 Initial 또는 Clean이면 Dirty로 전이하는지는 구현 정의이다. 벡터 상태도 유사하게 부정확하게 추적할 수 있으며, 소프트웨어가 VS=Initial 또는 VS=Clean을 쓰려 할 때 Dirty로 만들 수도 있다. VS가 Initial 또는 Clean이고 벡터 레지스터 또는 벡터 CSR에 쓰되 내용은 바꾸지 않는 명령어가 VS를 Dirty로 만드는지는 구현 정의이다.

표 4는 FS, VS, XS의 가능한 상태 전이를 모두 나타낸다. 표준 부동소수점 및 벡터 확장은 사용자 모드의 구성 해제나 비활성화/활성화 명령어를 지원하지 않는다.

표 4. FS와 VS 및 XS 상태 전이

동작	현재 Off	현재 Initial	현재 Clean	현재 Dirty
특권 코드의 문맥 저장	저장 안 함 → Off	저장 안 함 → Initial	저장 안 함 → Clean	저장함 → Clean
특권 코드의 문맥 복원	복원 안 함 → Off	초기 값으로 복원 → Initial	메모리에서 복원 → Clean	해당 없음
상태 읽기 명령어	예외 → Off	실행 → Initial	실행 → Clean	실행 → Dirty
구성 등 상태 변경 가능 명령어	예외 → Off	실행 → Dirty	실행 → Dirty	실행 → Dirty
장치 구성 해제 명령어	예외 → Off	실행 → Initial	실행 → Initial	실행 → Initial
장치 비활성화 명령어	실행 → Off	실행 → Off	실행 → Off	실행 → Off
장치 활성화 명령어	실행 → Initial	실행 → Initial	실행 → Initial	실행 → Initial

확장 상태를 불투명 객체로 취급하여 추가 상태의 세부 내용에서 특권 코드를 분리할 수 있도록, 확장 상태 초기화·저장·복원을 위한 표준 특권 명령어를 제공한다.

> **주석** 많은 보조 프로세서 확장은 제한된 문맥에서만 사용되므로, 사용이 끝나면 소프트웨어가 안전하게 구성 해제하거나 비활성화할 수 있다. 이는 큰 상태를 갖는 보조 프로세서의 문맥 교환 비용을 줄인다. 부동소수점 장치가 있으면 그 레지스터는 표준 호출 규약의 일부이므로 사용자 소프트웨어가 언제 안전하게 꺼도 되는지 알 수 없다. 따라서 부동소수점 상태를 다른 확장 상태와 구분한다.

XS는 모든 추가 확장 상태를 요약하지만, 확장 안에 추가 마이크로아키텍처 비트를 두어 문맥 저장·복원 비용을 더 줄일 수 있다.

SD는 읽기 전용이며 FS, VS, XS 중 하나라도 Dirty이면 설정된다. 즉 `SD = (FS==0b11 OR XS==0b11 OR VS==0b11)`이다. 따라서 특권 코드는 정수 레지스터 집합과 `pc` 외의 문맥 저장이 필요 없는 경우를 빠르게 판단할 수 있다.

부동소수점 상태는 항상 표준 F, D 및/또는 Q 명령어로 초기화·저장·복원한다. 특권 코드는 각 `f` 레지스터에 필요한 공간을 결정하기 위해 FLEN을 알아야 한다.

M모드와 S모드는 FS, VS, XS의 단일 복사본을 공유한다. 슈퍼바이저 소프트웨어는 보통 자신의 저장 문맥에 대한 상태를 직접 이 비트들에 기록한다. 머신 소프트웨어는 자신의 문맥 버전에서 확장 상태를 저장·복원할 때 더 보수적으로 처리해야 한다.

> **주석** 합리적인 사용 사례라면 사용자↔슈퍼바이저 문맥 교환 횟수가 다른 특권 수준으로의 문맥 교환 횟수보다 훨씬 많을 것이다. 비동기 인터럽트가 사용자 문맥 교환으로 이어지는 경우를 제외하면, 보조 프로세서는 비동기 인터럽트 처리를 위해 문맥 저장·복원을 요구해서는 안 된다.

2.1.1.6.8. mstatus의 이전 예상 랜딩 패드 ELP 상태

Zicfilp 확장은 이전 ELP를 저장하는 SPELP와 MPELP 필드를 추가한다. 이들은「트랩 시 예상 랜딩 패드 상태 보존」에 따라 갱신된다. xPELP 인코딩은 다음과 같다.

- 0 — NO_LP_EXPECTED: 랜딩 패드 명령어를 예상하지 않음.
- 1 — LP_EXPECTED: 랜딩 패드 명령어를 예상함.

2.1.1.7. 머신 트랩 벡터 기준 주소 레지스터 mtvec

mtvec는 벡터 기준 주소 BASE와 벡터 모드 MODE로 구성된 트랩 벡터 설정을 저장하는 MXLEN비트 WARL 읽기/쓰기 레지스터이다.

그림 9. mtvec MODE 필드 인코딩

비트 범위	필드
MXLEN-1:2	BASE[MXLEN-1:2] WARL
1:0	MODE WARL

mtvec는 항상 구현해야 하지만 읽기 전용 값을 포함할 수 있다. 쓰기 가능하면 저장할 수 있는 값의 집합은 구현마다 다를 수 있다. BASE는 항상 4바이트 경계에 정렬되어야 하며 MODE가 추가 정렬 제약을 부과할 수 있다. CSR에는 BASE 주소의 비트 XLEN-1:2만 들어 있다. 주소로 사용할 때는 하위 2비트를 0으로 채워 항상 4바이트 정렬된 XLEN비트 주소를 얻는다.

> **주석** 트랩 벡터 기준 주소 구현에는 상당한 유연성을 허용한다. 저사양 구현에 많은 상태 비트 부담을 주지 않으면서 대규모 시스템의 유연성도 허용하기 위해서이다.

표 5. mtvec MODE 필드 인코딩

값	이름	설명
0	Direct	모든 트랩에서 pc←BASE
1	Vectored	비동기 인터럽트에서 pc←BASE+4×cause
2 이상	—	예약됨

MODE=Direct이면 M모드로의 모든 트랩에서 pc를 BASE 주소로 설정한다. MODE=Vectored이면 M모드로의 모든 동기 예외에서 BASE로, 인터럽트에서는 BASE에 인터럽트 원인 번호의 4배를 더한 주소로 설정한다. 예를 들어 머신 타이머 인터럽트(표 6)는 pc=BASE+0x1c가 된다.

구현체는 모드별로 다른 정렬 제약을 둘 수 있다. 특히 Vectored는 Direct보다 더 엄격한 정렬을 요구할 수 있다.

> **주석** Vectored에서 더 큰 단위의 정렬을 허용하면 하드웨어 가산기 없이 벡터 처리를 구현할 수 있다.

리셋 및 NMI 벡터 위치는 플랫폼 명세가 정한다.

2.1.1.8. 머신 트랩 위임 레지스터 medeleg 및 mideleg

기본적으로 모든 특권 수준의 트랩은 머신 모드에서 처리한다. 다만 머신 처리기는 MRET(2.1.3.2절)로 적절한 수준에 트랩을 다시 보낼 수 있다. 성능을 높이기 위해 medeleg와 mideleg에 개별 읽기/쓰기 비트를 제공하여 특정 예외와 인터럽트를 낮은 특권 수준에서 직접 처리하도록 할 수 있다.

머신 예외 위임 레지스터 medeleg는 64비트 읽기/쓰기 레지스터이다. 머신 인터럽트 위임 레지스터 mideleg는 MXLEN비트 읽기/쓰기 레지스터이다. S모드가 있는 하트에는 두 레지스터가 존재해야 한다. 대응 비트를 설정하면 S

모드나 U모드에서 발생한 트랩을 S모드 처리기에 위임한다. S모드가 없는 하트에는 두 레지스터가 존재하지 않는 것이 바람직하다.

주석 버전 1.9.1 이전에는 M모드 전용 하트나 N 확장 없는 M/U 하트에서도 레지스터가 존재하되 0으로 고정되었다. `misa`로 존재 여부를 알 수 있으므로 이 경우 반드시 0을 반환하도록 요구할 이유가 없다.

트랩이 S모드로 위임되면 `scause`에는 원인, `sepc`에는 트랩한 명령어의 가상 주소, `stval`에는 예외별 데이터가 기록된다. `mstatus.SPP`에는 트랩 당시 특권 모드, SPIE에는 당시 SIE가 기록되고 SIE는 지워진다. `mcause`, `mepc`, `mtval` 및 `mstatus.MPP`, MPIE는 기록하지 않는다.

구현체는 위임 가능한 트랩의 일부만 지원할 수 있다. `medeleg`나 `mideleg`의 모든 비트에 1을 쓴 뒤 읽어, 1로 유지되는 위치를 확인하면 지원되는 위임 비트를 알 수 있다.

`medeleg`의 어떤 비트도 읽기 전용 1이어서는 안 된다. 즉 위임할 수 있는 동기 트랩은 위임하지 않는 것도 지원해야 한다. `mideleg`의 머신 인터럽트 대응 비트도 읽기 전용 1로 고정해서는 안 되지만, 하위 수준 인터럽트 비트는 그렇게 할 수 있다.

주석 버전 1.11 이전에는 `mideleg`의 어느 비트도 읽기 전용 1일 수 없었다. 플랫폼 표준은 언제든 그러한 제약을 추가할 수 있다.

트랩은 높은 특권 모드에서 낮은 모드로 전환하지 않는다. 예를 들어 M모드가 불법 명령어 예외를 S모드에 위임했더라도 M모드에서 불법 명령어를 실행하면 M모드에서 처리한다. 반면 수평 트랩은 가능하다. 같은 위임 설정에서 S모드가 불법 명령어를 실행하면 S모드에서 처리한다.

위임된 인터럽트는 위임한 특권 수준에서 마스킹된다. 예를 들어 `mideleg[5]`를 설정해 슈퍼바이저 타이머 인터럽트 STI를 S모드에 위임하면 M모드 실행 중에는 STI를 수신하지 않는다. 반대로 해당 비트가 0이면 어떤 모드에서도 STI를 수신할 수 있고 현재 모드와 관계없이 M모드로 제어를 넘긴다.

그림 10. 머신 예외 위임 레지스터 medeleg

비트 범위	필드
63:0	동기 예외 위임 WARL

`medeleg`는 표 6의 각 동기 예외마다 한 비트를 할당한다. 비트 인덱스는 `mcause`의 예외 번호와 같다. 예를 들어 비트 8을 설정하면 U모드 환경 호출을 낮은 특권 처리기에 위임할 수 있다. XLEN=32이면 `medelegh`가 `medeleg[63:32]`의 별칭인 32비트 읽기/쓰기 레지스터로 존재한다. XLEN=64이면 존재하지 않는다.

그림 11. 머신 인터럽트 위임 레지스터 mideleg

비트 범위	필드
MXLEN-1:0	인터럽트 WARL

`mideleg`는 개별 인터럽트의 트랩 위임 비트를 저장하며 비트 배치는 `mip`와 같다. 예를 들어 STIP 위임 제어는 비트 5이다. 낮은 특권 모드에서 발생할 수 없는 예외의 `medeleg` 비트는 읽기 전용 0인 것이 바람직하다. 특히 `medeleg[11]`은 읽기 전용 0이다. 이중 트랩은 위임할 수 없으므로 `medeleg[16]`도 읽기 전용 0이다.

2.1.1.9. 머신 인터럽트 레지스터 mip 및 mie

`mip`는 대기 중인 인터럽트 정보를 담는 MXLEN비트 읽기/쓰기 레지스터이고, `mie`는 대응 활성화 비트를 담는 MXLEN비트 읽기/쓰기 레지스터이다. `mcause`에 보고되는 인터럽트 원인 번호 i는 두 레지스터의 비트 i에 대응한다. 비트 15:0은 표준 인터럽트 원인 전용이고, 16 이상은 플랫폼 용도로 지정된다.

주석 플랫폼은 재량에 따라 플랫폼용 인터럽트를 사용자 정의 용도로 지정할 수 있다.

그림 12. 머신 인터럽트 대기 레지스터 mip

비트 범위	필드
MXLEN-1:0	인터럽트 WARL

그림 13. 머신 인터럽트 활성화 레지스터 mie

비트 범위	필드
MXLEN-1:0	인터럽트 WARL

다음 조건이 모두 참이면 인터럽트 i가 M모드로 트랩하여 특권 모드가 M으로 바뀐다. (a) 현재 모드가 M이고 `mstatus.MIE=1`이거나 현재 모드가 M보다 낮다. (b) `mip[i]`와 `mie[i]`가 모두 1이다. (c) `mideleg`가 있으면 해당 비트 i가 0이다.

이 조건들은 `mip`에서 인터럽트가 대기 상태가 되거나 해제된 시점부터 유한 시간 안에 평가되어야 한다. 또한 xRET 실행 직후, 또는 `mip`, `mie`, `mstatus`, `mideleg` 등 이 조건들이 명시적으로 의존하는 CSR에 명시적으로 쓴 직후에도 평가해야 한다.

M모드 인터럽트는 더 낮은 모드의 모든 인터럽트보다 우선한다. `mip`의 각 비트는 쓰기 가능 또는 읽기 전용일 수 있다. 쓰기 가능한 `mip[i]`에 0을 쓰면 인터럽트 i의 대기 상태를 지울 수 있다. 인터럽트 i가 대기 상태가 될 수 있지만 대응 비트가 읽기 전용이면, 구현체는 다른 해제 메커니즘을 제공해야 한다. 대기 상태가 될 수 있는 인터럽트의 `mie` 비트는 쓰기 가능해야 한다. 쓰기 불가능한 `mie` 비트는 읽기 전용 0이어야 한다.

`mip`와 `mie`의 표준 부분인 비트 15:0의 형식은 각각 그림 14와 그림 15와 같다.

그림 14. mip의 표준 부분 비트 15:0

비트 범위	필드
15:14	0
13	LCOFIP
12	0
11	MEIP
10	0
9	SEIP
8	0
7	MTIP
6	0
5	STIP
4	0
3	MSIP
2	0
1	SSIP
0	0

그림 15. mie의 표준 부분 비트 15:0

비트 범위	필드
15:14	0
13	LCOFIE
12	0
11	MEIE
10	0
9	SEIE
8	0
7	MTIE
6	0
5	STIE
4	0
3	MSIE
2	0
1	SSIE
0	0

주석 머신 인터럽트 레지스터는 단순화를 위해 서비스 우선순위가 고정된 소수의 최상위 인터럽트 소스를 다룬다. 별도의 외부 인터럽트 제어기는 훨씬 많은 인터럽트에 복잡한 우선순위를 적용한 뒤 머신 인터럽트 소스로 다중화할 수 있다.

NMI는 NMI 처리기를 실행한다는 사실로 존재를 암묵적으로 알 수 있으므로 `mip`에 나타내지 않는다.

`mip.MEIP`와 `mie.MEIE`는 머신 외부 인터럽트 대기 및 활성화 비트이다. MEIP는 `mip`에서 읽기 전용이며 플랫폼별 인터럽트 제어기가 설정·해제한다. `mip.MTIP`와 `mie.MTIE`는 머신 타이머 인터럽트 대기 및 활성화 비트이다. MTIP는 읽기 전용이며 메모리 매핑 머신 타이머 비교 레지스터에 써서 지운다.

`mip.MSIP`와 `mie.MSIE`는 머신 소프트웨어 인터럽트 대기 및 활성화 비트이다. MSIP는 `mip`에서 읽기 전용이며, 머신 수준 프로세서 간 인터럽트를 제공하는 메모리 매핑 제어 레지스터 접근으로 기록한다. 하트의 메모리 매핑 `msip`는 32비트 읽기/쓰기 레지스터이다. 비트 31:1은 0으로 읽히고 비트 0에 MSIP가 있다. `msip` 변경은 즉시가 아닐 수 있지만 결국 `mip.MSIP`에 반영되어야 한다. 단일 하트 시스템이거나 플랫폼 표준이 프로세서 간 인터럽트를 MEI로 전달하도록 지원하면 `mip.MSIP`와 `mie.MSIE`는 모두 읽기 전용 0일 수 있다.

S모드를 구현하지 않으면 `mip`의 SEIP, STIP, SSIP와 `mie`의 SEIE, STIE, SSIE는 읽기 전용 0이다.

S모드를 구현하면 SEIP와 SEIE는 슈퍼바이저 외부 인터럽트 대기 및 활성화 비트이다. SEIP는 쓰기 가능하며 M모드 소프트웨어가 S모드에 외부 인터럽트 대기를 알리도록 쓸 수 있다. 플랫폼 수준 인터럽트 제어기도 슈퍼바이저 외부 인터럽트를 생성할 수 있다. 실제 대기는 소프트웨어 쓰기 가능 SEIP와 외부 제어기 신호의 논리 OR로 결정된다. CSR 명령어로 `mip`를 읽어 `rd`로 반환할 때는 이 OR값을 사용하지만, SEIP에 쓸 값을 계산할 때는 외부 제어기 신호를 사용하지 않는다. CSRRS/CSRRC의 읽기-수정-쓰기에는 소프트웨어 쓰기 가능 SEIP만 참여한다.

주석 소프트웨어 비트를 B, 외부 신호를 E라고 하자. `csrrs t0, mip, t1`은 `t0[9]←B || E, B←B || t1[9]`로 동작한다. `csrrw t0, mip, t1`은 `t0[9]←B || E, B←t1[9]`로 동작한다. 어느 경우에도 B는 E에 의존하지 않는다. 이는 실제 외부 인터럽트를 잃지 않으면서 높은 특권 계층이 외부 인터럽트를 깔끔하게 모사하도록 한 설계이며, 결과적으로 일반 CSR 접근과 비교해 CSR 명령어 동작이 약간 달라진다.

S모드를 구현하면 STIP와 STIE는 슈퍼바이저 타이머 인터럽트 대기 및 활성화 비트이다. `stimecmp`를 구현하지 않으면 STIP는 `mip`에서 쓰기 가능하며 M모드 소프트웨어가 S모드에 타이머 인터럽트를 전달할 수 있다. `stimecmp`를 구현하면 STIP는 읽기 전용이며 `stimecmp`가 만드는 슈퍼바이저 타이머 인터럽트 신호를 반영한다. 이 신호는 현재 시간보다 큰 값을 `stimecmp`에 써서 지운다.

S모드를 구현하면 SSIP와 SSIE는 슈퍼바이저 소프트웨어 인터럽트 대기 및 활성화 비트이다. SSIP는 `mip`에서 쓰기 가능하며 플랫폼별 인터럽트 제어기가 1로 설정할 수도 있다.

Sscofpmf를 구현하면 LCOFIP와 LCOFIE는 로컬 카운터 오버플로 인터럽트 대기 및 활성화 비트이다. LCOFIP는 읽기/쓰기 가능하며, 어느 `mhpmeventn.OF` 비트든 설정되어 발생한 로컬 카운터 오버플로 요청을 반영한다. Sscofpmf가 없으면 두 비트는 읽기 전용 0이다.

M모드를 대상으로 동시에 발생한 인터럽트의 우선순위는 높은 순서대로 MEI, MSI, MTI, SEI, SSI, STI, LCOFI이다.

> **주석** 선점 지원을 위해 높은 특권 모드의 인터럽트를 낮은 모드보다 먼저 처리한다. 비트 16 이상 플랫폼별 머신 인터럽트의 우선순위는 플랫폼이 정하지만, 매우 빠른 로컬 벡터 인터럽트를 지원하려고 보통 최상위로 둔다. 외부 인터럽트는 응답 지연이 짧아야 하는 장치에서 오는 경우가 많아 내부 타이머·소프트웨어 인터럽트보다 먼저 처리한다. 소프트웨어 인터럽트는 프로세서 간 메시징에, 내부 타이머는 정밀도가 덜 중요한 시간 분할에 주로 쓰이므로 소프트웨어가 타이머보다 우선한다. 고정밀 타이밍이 필요하면 소프트웨어 인터럽트를 피하거나 고정밀 타이머를 다른 인터럽트 경로로 보낼 수 있다. 소프트웨어 인터럽트를 `mip`의 하위 4비트에 배치한 것은 소프트웨어 쓰기가 잦고 5비트 즉시값 CSR 명령어 하나로 처리할 수 있기 때문이다.

슈퍼바이저 수준의 `sip`와 `sie`는 `mip`와 `mie`의 제한된 뷰이다. `mideleg` 비트로 인터럽트를 S모드에 위임하면 `sip`에서 보이고 `sie`로 마스킹할 수 있다. 그렇지 않으면 `sip`와 `sie`의 대응 비트는 읽기 전용 0이다.

2.1.1.10. 하드웨어 성능 모니터

M모드에는 기본적인 하드웨어 성능 모니터링 기능이 있다. `mcycle` CSR은 해당 하트가 실행되는 프로세서 코어의 클록 사이클 수를 세고, `minstret`는 하트가 완료(retire)한 명령어 수를 센다. 두 레지스터는 모든 RV32 및 RV64 하트에서 64비트 정밀도를 갖는다.

카운터의 리셋 직후 값은 임의이며 원하는 값을 쓸 수 있다. CSR 쓰기는 쓰기를 수행하는 명령어의 다른 동작이 모두 완료된 뒤 효력이 발생한다. 같은 코어의 하트들이 `mcycle`을 공유할 수 있으며, 이때 쓰기 결과는 공유 하트들에 보인다. 플랫폼은 어떤 하트가 `mcycle`을 공유하는지 알려 주는 메커니즘을 제공하는 것이 바람직하다.

하드웨어 성능 모니터에는 추가 64비트 이벤트 카운터 `mhpmcounter3~mhpmcounter31` 29개가 있다. 이벤트 선택 CSR `mhpmevent3~mhpmevent31`은 대응 카운터를 증가시키는 이벤트를 제어하는 64비트 WARL 레지스터이다. 이벤트 의미는 플랫폼이 정의하지만 이벤트 0은 '이벤트 없음'이다. 모든 카운터를 구현하는 것이 바람직하나, 카운터와 대응 이벤트 선택기를 모두 읽기 전용 0으로 만드는 것도 유효한 구현이다.

그림 16. 하드웨어 성능 모니터 카운터 각 필드는 64비트

카운터 63:0	이벤트 선택기 63:0
mcycle	—
minstret	—
mhpmcounter3	mhpmevent3
mhpmcounter4	mhpmevent4
⋮	⋮
mhpmcounter30	mhpmevent30

카운터 63:0	이벤트 선택기 63:0
mhpmcounter31	mhpmevent31

mhpmcounter는 RV32와 RV64에서 최대 64비트 정밀도를 지원하는 WARL 레지스터이다. XLEN=32일 때 mcycle, minstret, mhpmcounterₙ, mhpmeventₙ을 읽으면 대응 레지스터의 비트 31:0을 반환하고 쓰기도 그 부분만 바꾼다. mcycleh, minstreth, mhpmcounterₙh, mhpmeventₙh는 비트 63:32를 읽고 쓴다. mhpmeventₙh CSR은 Sscofpmf 확장을 구현한 경우에만 제공된다.

2.1.1.11. 머신 카운터 활성화 레지스터 mcounteren

mcounteren은 바로 아래 특권 모드에서 하드웨어 성능 카운터를 사용할 수 있는지를 제어하는 32비트 레지스터이다.

그림 17. 카운터 활성화 레지스터 mcounteren

비트 범위	필드
31:3	HPM31:HPM3 각 비트는 대응 카운터 제어
2	IR
1	TM
0	CY

이 레지스터는 접근 가능 여부만 제어한다. 읽거나 써도 실제 카운터에는 영향을 주지 않으며, 접근할 수 없는 동안에도 카운터는 계속 증가한다.

CY, TM, IR, HPMn이 0이면 S모드 또는 U모드에서 각각 cycle, time, instret, hpmcounterₙ을 읽으려 할 때 불법 명령어 예외가 발생한다. 비트가 1이면 바로 아래에 구현된 특권 모드, 즉 S가 있으면 S모드, 아니면 U모드에서 해당 레지스터에 접근할 수 있다.

> **주석** 이 비트는 적은 하드웨어로 두 사용 사례를 지원한다. 고성능 타이머·카운터가 필요 없는 하트에서는 머신 소프트웨어가 접근을 트랩하고 모두 소프트웨어로 구현할 수 있다. 고성능이 필요하고 실제 하드웨어 카운터를 숨길 필요가 없으면 낮은 특권 모드에 직접 노출할 수 있다.

추가로 TM=0이면 M보다 낮은 모드에서 stimecmp 또는 vstimecmp에 접근할 때 불법 명령어 예외가 발생한다. TM=1이면 S모드가 구현된 경우 S모드에서 두 레지스터에 접근할 수 있다. VS모드가 구현되어 있고 hcounteren.TM이 금지하지 않으면 VS모드는 stimecmp를 통해 vstimecmp에 접근할 수 있다.

cycle, instret, hpmcounterₙ은 각각 mcycle, minstret, mhpmcounterₙ의 읽기 전용 ㅅㅐ도이다. time은 메모리 매핑 mtime의 읽기 전용 ㅅㅐ도이다. XLEN=32이면 cycleh, instreth, hpmcounterₙh는 대응 머신 카운터의 상위 절반 ㅅㅐ도이고, timeh는 mtime[63:32]의 ㅅㅐ도이며 time은 mtime[31:0]만 나타낸다.

> **주석** 구현체는 time·timeh 읽기를 메모리 매핑 mtime 로드로 바꾸거나, M모드 소프트웨어가 낮은 특권 모드를 대신해 이 기능을 에뮬레이션하게 할 수 있다.

U모드가 있는 하트에는 mcounteren을 구현해야 한다. 다만 모든 필드가 WARL이며 읽기 전용 0일 수 있고, 이는 낮은 특권 모드의 대응 카운터 읽기가 불법 명령어 예외를 발생시킴 을 뜻한다. U모드가 없는 하트에는 이 레지스터가 존재하지 않는 것이 바람직하다.

2.1.1.12. 머신 카운터 증가 억제 레지스터 mcountinhibit

그림 18. 카운터 증가 억제 레지스터 mcountinhibit

비트 범위	필드
31:3	HPM31:HPM3 각 비트는 대응 카운터 제어
2	IR
1	0
0	CY

mcountinhibit는 하드웨어 성능 카운터 중 어떤 것이 증가할지를 제어하는 32비트 WARL 레지스터이다. 증가 여부만 제어하며 접근 가능 여부에는 영향을 주지 않는다. CY, IR, HPMn이 0이면 각각 mcycle, minstret, mhpmcounterₙ은 평소대로 증가한다. 비트가 1이면 대응 카운터는 증가하지 않는다.

같은 코어의 하트들이 mcycle을 공유하면 mcountinhibit.CY도 공유하며, 이 필드에 대한 쓰기는 그 하트들에 보인다. mcountinhibit를 구현하지 않으면 값이 0인 것처럼 동작한다.

> **주석** mcycle과 minstret가 필요하지 않을 때는 조건부로 증가를 억제하여 에너지를 줄이는 것이 바람직하다. 한 CSR로 모든 카운터를 억제하면 원자적 샘플링도 가능하다. mtime은 여러 코어가 공유할 수 있으므로 이 메커니즘으로 억제할 수 없다.

2.1.1.13. 머신 임시 저장 레지스터 mscratch

mscratch는 M모드 전용 MXLEN비트 읽기/쓰기 레지스터이다. 일반적으로 하트별 M모드 문맥 공간 포인터를 저장하고, M모드 트랩 처리기에 진입할 때 사용자 레지스터와 교환하는 데 사용한다.

그림 19. 머신 모드 임시 저장 레지스터

비트 범위	필드
MXLEN-1:0	mscratch

> **주석** MIPS ISA는 OS용 사용자 레지스터 k0, k1을 할당했다. 빠르고 단순하지만 사용 가능한 사용자 레지스터를 줄이고 추가 특권 수준이나 중첩 트랩으로 확장하기 어렵다. 잠재적 보안 취약점을 피하고 결정적인 디버깅 동작을 제공하려고 사용자 수준 복귀 전에 두 레지스터를 지워야 할 수도 있다. RISC-V 사용자 ISA는 다양한 특권 시스템 환경을 지원하도록 설계되었으므로 OS 의존 기능을 넣지 않으려 했다. CSR 교환 명령어는 mscratch 값을 빠르게 저장·복원한다. MIPS와 달리 OS는 사용자 문맥 실행 중에도 mscratch에 값이 유지됨을 신뢰할 수 있다.

2.1.1.14. 머신 예외 프로그램 카운터 레지스터 mepc

mepc는 그림 20 형식의 MXLEN비트 읽기/쓰기 레지스터이다. mepc[0]은 항상 0이다. IALIGN=32만 지원하는 구현에서는 mepc[1:0]이 항상 0이다. misa 변경 등으로 IALIGN=16 또는 32를 허용하면, IALIGN=32일 때 mepc[1]은 읽기에서 마스킹되어 0으로 보인다. MRET의 암묵적 읽기도 마찬가지이다. 마스킹되어도 IALIGN=32에서 mepc[1]은 여전히 쓰기 가능하다.

mepc는 모든 유효 가상 주소를 담을 수 있어야 하는 WARL 레지스터이다. 가능한 모든 무효 주소를 담을 필요는 없다. 쓰기 전에 무효 주소를 mepc가 담을 수 있는 다른 무효 주소로 바꿀 수 있다.

> **주석** 주소 변환이 적용되지 않으면 가상 주소와 물리 주소가 같다. 따라서 mepc가 표현해야 하는 주소 집합에는 유효한 pc 또는 유효 주소로 사용할 수 있는 물리 주소도 포함된다.

M모드로 트랩하면 mepc에 중단되거나 예외를 일으킨 명령어의 가상 주소를 기록한다. 그 밖에는 구현체가 자동으로 쓰지 않지만 소프트웨어가 명시적으로 쓸 수 있다.

그림 20. 머신 예외 프로그램 카운터 레지스터

비트 범위	필드
MXLEN-1:0	mepc

2.1.1.15. 머신 원인 레지스터 mcause

mcause는 그림 21 형식의 MXLEN비트 읽기/쓰기 레지스터이다. M모드로 트랩하면 트랩 원인 사건의 코드를 기록한다. 그 밖에는 구현체가 자동으로 쓰지 않지만 소프트웨어가 명시적으로 쓸 수 있다.

인터럽트로 트랩했으면 Interrupt 비트를 설정한다. Exception Code 필드는 마지막 예외 또는 인터럽트의 식별 코드이다. 가능한 머신 예외 코드는 표 6과 같다. 이 필드는 WLRL이므로 지원되는 예외 코드만 저장할 수 있다고 보장한다.

그림 21. 머신 원인 레지스터 mcause

비트 범위	필드
MXLEN-1	Interrupt
MXLEN-2:0	Exception Code WLRL

로드와 LR은 로드 예외를 발생시키고, 스토어·SC·AMO는 스토어/AMO 예외를 발생시킨다.

> **주석** mcause 값의 부호에 대한 분기 하나로 인터럽트를 다른 트랩과 구분할 수 있다. 왼 쪽 시프트로 인터럽트 비트를 제거하면서 예외 코드를 배율 조정하여 트랩 벡 터 테이블의 인덱 스로 사용할 수 있다.
>
> 특권 명령어 예외와 불법 명령어 예외를 구분하지 않는다. 이는 아키텍처를 단순화하고 어떤 상위 특권 명령어를 지원하는지 숨 긴다. 트랩을 처리하는 특권 수준은 둘을 구분할지, 구분한다면 해당 연산 코드를 불법 또는 특권 명령어로 취급할지 정책을 정할 수 있다.

한 명령어가 여러 동기 예외를 일으킬 수 있으면 표 7의 우선순위에 따라 하나를 수신하여 mcause에 보고한다. 사용자 정의 동기 예외의 우선순위는 구현 정의이다.

표 6. 트랩 후 mcause 레지스터 값

Interrupt	예외 코드	설명
1	0	예약됨
1	1	슈퍼바이저 소프트웨어 인터럽트
1	2	예약됨
1	3	머신 소프트웨어 인터럽트
1	4	예약됨
1	5	슈퍼바이저 타이머 인터럽트
1	6	예약됨
1	7	머신 타이머 인터럽트
1	8	예약됨
1	9	슈퍼바이저 외부 인터럽트
1	10	예약됨
1	11	머신 외부 인터럽트

Interrupt	예외 코드	설명
1	12	예약됨
1	13	카운터 오버플로 인터럽트
1	14~15	예약됨
1	16 이상	플랫폼 용도
0	0	명령어 주소 비정렬
0	1	명령어 접근 폴트
0	2	불법 명령어
0	3	중단점
0	4	로드 주소 비정렬
0	5	로드 접근 폴트
0	6	스토어/AMO 주소 비정렬
0	7	스토어/AMO 접근 폴트
0	8	U모드 환경 호출
0	9	S모드 환경 호출
0	10	예약됨
0	11	M모드 환경 호출
0	12	명령어 페이지 폴트
0	13	로드 페이지 폴트
0	14	예약됨
0	15	스토어/AMO 페이지 폴트
0	16	이중 트랩
0	17	예약됨
0	18	소프트웨어 검사
0	19	하드웨어 오류
0	20~23	예약됨
0	24~31	사용자 정의 용도
0	32~47	예약됨
0	48~63	사용자 정의 용도
0	64 이상	예약됨

표 7. 동기 예외 우선순위 높은 순서

우선순위	예외 코드	설명
최고	3	명령어 주소 중단점
↓	12, 1	명령어 주소 변환 중 처음 만난 페이지 폴트 또는 접근 폴트
↓	1	명령어 물리 주소에 대한 명령어 접근 폴트

우선순위	예외 코드	설명
↓	2	불법 명령어
↓	0	명령어 주소 비정렬
↓	8, 9, 11	환경 호출
↓	3	환경 중단 EBREAK
↓	3	로드/스토어/AMO 주소 중단점
↓	4, 6	선택적으로 로드/스토어/AMO 주소 비정렬
↓	13, 15, 5, 7	명시적 메모리 접근의 주소 변환 중 처음 만난 페이지 폴트 또는 접근 폴트
↓	5, 7	명시적 메모리 접근의 물리 주소에 대한 로드/스토어/AMO 접근 폴트
최저	4, 6	더 높은 우선순위로 처리하지 않은 로드/스토어/AMO 주소 비정렬

가상 주소를 물리 주소로 변환할 때 어떤 예외가 발생하는지는 주소 변환 알고리즘이 결정한다. 로드/스토어/AMO 주소 비정렬 예외는 페이지 폴트 및 접근 폴트보다 우선순위가 높을 수도 낮을 수도 있다.

> **주석** 이 상대적 우선순위는 두 설계 유형을 지원하려고 구현 정의로 두었다. 비정렬 접근을 전혀 지원하지 않는 구현은 주소 변환이나 보호 검사 없이 비정렬 예외를 발생시킬 수 있다. 일부 물리 주소에서만 비정렬 접근을 지원하는 구현은 허용 여부를 판단하기 전에 주소를 변환·검사해야 하므로 페이지 폴트 또는 접근 폴트를 발생시키는 것이 더 적절하다.
>
> 명령어 주소 중단점은 데이터 주소 중단점(워치포인트) 및 EBREAK가 만드는 환경 중단 예외와 원인 값은 같지만 우선순위가 다르다.
>
> 명령어 주소 비정렬 예외는 인출 자체가 아니라 비정렬 목적지를 갖는 제어 흐름 명령어에서 발생한다. 따라서 다른 명령어 주소 예외보다 우선순위가 낮다.
>
> **주석** 소프트웨어 검사 예외는 제어 흐름 및 메모리 접근 제약 등 소프트웨어 자산의 무결성을 보호하는 ISA 확장의 검사·단언을 위반할 때 발생하는 동기 예외이다. `xtval`에는 0 또는 그 예외를 규정한 확장이 정의한 정보 값이 기록된다. 다른 동기 예외와의 상대 우선순위는 발생 원인에 따라 달라지며 해당 확장이 정한다. 하드웨어 오류 예외는 명령어가 손상되거나 수정 불가능한 데이터에 명시적 또는 암묵적으로 접근할 때 발생하는 동기 예외이다. 여기서 데이터는 하트 안에서 사용하는 모든 종류의 정보를 포함한다.

하드웨어 오류 예외에서는 `xepc`에 손상 데이터 접근을 시도한 명령어 주소를 기록하고, `xtval`에는 0 또는 손상 데이터에 접근한 명령어 인출·로드·스토어의 가상 주소를 기록한다. 우선순위는 구현 정의이지만, 보통 전체 우선순위 순서 중 하드웨어 오류가 발견되는 지점에서 해당 발생을 인식할 것으로 예상한다.

2.1.1.16. 머신 트랩 값 레지스터 mtval

`mtval`은 그림 22 형식의 MXLEN비트 읽기/쓰기 레지스터이다. M모드로 트랩하면 0 또는 소프트웨어의 트랩 처리를 돕는 예외별 정보를 기록한다. 그 밖에는 구현체가 자동으로 쓰지 않지만 소프트웨어가 명시적으로 쓸 수 있다.

플랫폼은 어떤 예외가 유의미한 정보를 기록해야 하는지, 어떤 예외가 항상 0을 쓸 수 있는지, 어떤 예외가 원인 사건에 따라 두 동작 중 하나를 보일 수 있는지 정한다. 어떤 예외도 0이 아닌 값을 쓰지 않도록 정한 플랫폼에서는 `mtval`이 읽기 전용 0이다.

명령어 인출·로드·스토어에서 중단점, 주소 비정렬, 접근 폴트, 페이지 폴트, 하드웨어 오류 예외가 발생하고 `mtval`에 0이 아닌 값을 쓰면, 그 값은 폴트를 일으킨 가상 주소이다. EBREAK 또는 C.EBREAK의 중단점 예외에서는 0 또는 명령어의 가상 주소를 기록한다.

> **주석** [C.]EBREAK의 명령어 주소는 이미 `mepc`에 있으므로 `mtval`에 같은 주소를 기록하는 것은 중복이다. 하위 호환성을 위해 선택을 허용한다.

페이지 기반 가상 메모리를 활성화하면 물리 메모리 접근 폴트에서도 mtval에 폴트 가상 주소를 기록한다. 이는 특히 하드웨어 페이지 테이블 워커를 갖는 구현의 데이터 경로 비용을 줄인다.

그림 22. 머신 트랩 값 레지스터

비트 범위	필드
MXLEN-1:0	mtval

비정렬 로드·스토어가 접근 폴트, 페이지 폴트 또는 하드웨어 오류를 일으키고 mtval에 0이 아닌 값을 쓰면, 실패한 접근 부분의 가상 주소를 기록한다. 가변 길이 명령어를 지원하는 하트의 명령어 접근 폴트, 페이지 폴트 또는 하드웨어 오류에서 0이 아닌 값을 쓰면, mtval은 실패한 명령어 부분의 가상 주소이고 mepc는 명령어 시작 주소이다.

선택적으로 불법 명령어 예외에서 실패한 명령어 비트를 mtval에 반환할 수도 있다. mepc는 메모리의 해당 명령어를 가리킨다. 이때 0이 아닌 값을 쓴다면 다음 중 가장 짧은 것을 담는다.

- 실제로 실패한 명령어 전체.
- 해당 명령어의 처음 ILEN비트.
- 해당 명령어의 처음 MXLEN비트.

값은 오른쪽 정렬하며 사용하지 않는 상위 비트는 모두 0으로 지운다.

> **주석** 실패 명령어를 mtval에 담으면 에뮬레이션 비용을 줄일 수 있다. 비정렬 명령어를 여러 번 부분 로드할 필요를 피할 수 있고, 데이터 레지스터로 명령어를 로드할 때 생길 수 있는 데이터 캐시 미스나 느린 비캐시 접근도 줄인다. 동적 변환 시스템처럼 다른 에이전트가 명령어 메모리를 바꿀 때의 원자성 문제도 있다. 트랩 전에 명령어 전체 또는 적어도 처음 MXLEN비트를 mtval에 담아야 한다. 보통 디코딩 전에 명령어 전체를 인출하므로 구현을 제약하지 않으며 소프트웨어 처리기도 단순해진다. mtval=0이면 기능을 지원하지 않거나 불법인 0 명령어를 인출한 경우이다. mepc의 명령어 메모리를 로드하면 둘을 구분할 수 있다. 또는 실행 전에 시스템 구성 정보를 조회하여 적절한 처리기를 설치할 수 있다.

소프트웨어 검사 예외에 의한 트랩에서는 mtval에 다음 예외 원인 인코딩을 담는다.

- 0 — 정보 없음.
- 2 — 랜딩 패드 폴트. Zicfilp 확장 정의.
- 3 — 섀도 스택 폴트. Zicfiss 확장 정의.

그 밖의 트랩에서는 0으로 설정하지만 향후 표준이 다른 트랩의 설정을 재정의할 수 있다. 읽기 전용 0이 아니라면 mtval은 모든 유효 가상 주소와 0을 담아야 하는 WARL 레지스터이다. 모든 무효 주소까지 담을 필요는 없으며, 쓰기 전에 표현 가능한 다른 무효 주소로 바꿀 수 있다. 실패 명령어 비트 반환 기능을 구현하면 2^N보다 작은 모든 값도 담을 수 있어야 한다. N은 MXLEN과 ILEN 중 작은 값이다.

2.1.1.17. 머신 구성 포인터 레지스터 mconfigptr

mconfigptr는 그림 23 형식의 MXLEN비트 읽기 전용 CSR이며 구성 자료구조의 물리 주소를 담는다. 소프트웨어는 그 자료구조를 순회하여 하트, 플랫폼 및 구성 정보를 확인할 수 있다.

그림 23. 머신 구성 포인터 레지스터

비트 범위	필드
MXLEN-1:0	mconfigptr

포인터의 정렬 단위는 비트 수로 MXLEN 이상이어야 한다. 즉 MXLEN=8×n이면 mconfigptr[$\log_2(n)$-1:0]은 0이어야 한다. 레지스터는 반드시 구현하지만, 자료구조가 없거나 다른 방식으로 찾아야 함을 나타내기 위해 0일 수 있다.

주석 구성 자료구조의 형식과 스키마는 아직 표준화되지 않았다.

일부 구현은 `mconfigptr`를 상수로 고정하지만, CSR 읽기가 반환하는 값을 설정할 수 있는 구현도 있다. 예를 들어 부팅 초기에 플랫폼이나 M모드 소프트웨어가 설정하는 메모리 매핑 레지스터 값을 `mconfigptr`가 보여 줄 수 있다.

2.1.1.18. 머신 환경 설정 레지스터 menvcfg

`menvcfg`는 그림 24 형식의 64비트 읽기/쓰기 CSR이며 M보다 낮은 모드의 실행 환경에 관한 특정 특성을 제어한다.

그림 24. 머신 환경 설정 레지스터 menvcfg

비트 범위	필드
63	STCE
62	PBMTE
61	ADUE
60	CDE
59	DTE
58:34	WPRI
33:32	PMM
31:8	WPRI
7	CBZE
6	CBCFE
5:4	CBIE
3	SSE
2	LPE
1	WPRI
0	FIOM

FIOM(Fence of I/O implies Memory)이 1이면 낮은 특권 모드의 FENCE에서 장치 I/O 접근 순서를 보장하라는 요구가 주 메모리 접근 순서 보장도 포함하도록 바꾼다. 이때 PI, PO, SI, SO의 해석은 표 8과 같다. 마찬가지로 FIOM=1인 낮은 특권 모드에서 장치 I/O로 순서화되는 영역에 접근하는 원자적 명령어의 aq 및/또는 rl비트가 설정되면, 그 명령어는 장치 I/O와 메모리를 모두 접근하는 것처럼 순서화된다.

S모드를 지원하지 않거나 `satp.MODE`가 읽기 전용 0으로 항상 Bare이면 FIOM을 읽기 전용 0으로 만들 수 있다.

표 8. FIOM이 1일 때 M보다 낮은 모드의 FENCE 선행 및 후행 집합 해석

명령어 비트	비트가 1일 때의 의미
PI	선행 장치 입력 및 메모리 읽기 PR도 포함
PO	선행 장치 출력 및 메모리 쓰기 PW도 포함
SI	후행 장치 입력 및 메모리 읽기 SR도 포함
SO	후행 장치 출력 및 메모리 쓰기 SW도 포함

주석 H 확장의 하이퍼바이저 CSR `henvcfg`에는 같은 FIOM비트가 있으므로, M모드에서 H 확장을 에뮬레이션할 수 있도록 `menvcfg`에도 FIOM이 필요하다.

PBMTE는 S모드 및 G단계 주소 변환, 즉 `satp` 또는 `hgatp`가 가리키는 페이지 테이블에서 Svpbmt를 사용할 수 있는지 제어한다. PBMTE=1이면 사용할 수 있고 0이면 Svpbmt가 구현되지 않은 것처럼 동작한다. Svpbmt가 없으면 읽기 전용 0이다. H 확장이 있더라도 `menvcfg.PBMTE=0`이면 `henvcfg.PBMTE`는 읽기 전용 0이다. PBMTE 변경 후 `rs1=x0`, `rs2=x0`인 SFENCE.VMA를 실행하면 PTE의 PBMT 필드 해석 변경에 대해 주소 변환 캐시를 동기화하기에 충분하다. H 확장의 추가 동기화 요구사항은「메모리 관리 펜스」를 참조한다.

Svadu를 구현하면 ADUE가 S모드 및 G단계 주소 변환 중 PTE A/D비트의 하드웨어 갱신을 제어한다. ADUE=1이면 S모드 변환에서 하드웨어 갱신을 활성화하고 Svade가 구현되지 않은 것처럼 동작한다. H 확장이 있고 ADUE=1이면 G단계에서도 같은 방식으로 동작한다. ADUE=0이면 S모드와 G단계에서 Svade가 구현된 것처럼 동작한다. Svadu가 없으면 ADUE는 읽기 전용 0이다. H 확장이 있어도 `menvcfg.ADUE=0`이면 `henvcfg.ADUE`는 읽기 전용 0이다. ADUE 변경 후 `rs1=x0`, `rs2=x0`인 SFENCE.VMA는 PTE A/D비트 해석 변경에 대한 캐시 동기화에 충분하다. H 확장의 추가 요구는「메모리 관리 펜스」를 참조한다.

주석 Svade는 PTE A/D비트를 설정해야 할 때 페이지 폴트를 요구한다. 따라서 ADUE=0일 때 Svade가 구현된 상태이다.

Smcdeleg를 구현하면 CDE(Counter Delegation Enable)가 Zicntr 및 Zihpm 카운터를 S모드에 위임할 수 있는지 제어한다. CDE=1이면 Smcdeleg가 활성화된다. 자세한 내용은「Smcdeleg/Ssccfg 카운터 위임 확장」을 참조한다. CDE=0이면 두 확장이 구현되지 않은 것처럼 보인다. Smcdeleg가 없으면 CDE는 읽기 전용 0이다.

Sstc는 STCE(STimecmp Enable)를 추가한다. Sstc가 없으면 STCE는 읽기 전용 0이다. STCE=1이면 S모드에서 `stimecmp`를 사용할 수 있다. Sstc를 구현했어도 `menvcfg.STCE=0`이면 M 이외의 모드에서 `stimecmp` 접근 시 불법 명령어 예외가 발생하고, `henvcfg.STCE`는 읽기 전용 0이 되며 `mip·sip`의 STIP는 확장이 없는 경우의 정의된 동작으로 돌아간다. H 확장이 있으면 `hip.VSTIP`도 확장이 없는 경우의 동작으로 돌아간다.

Zicboz는 CBZE(Cache Block Zero instruction Enable)를 추가한다. CBZE=1이면 M보다 낮은 모드에서 `CBO.ZERO` 실행을 허용하고, 아니면 불법 명령어 예외를 발생시킨다. Zicboz가 없으면 CBZE는 읽기 전용 0이다.

Zicbom은 CBCFE(Cache Block Clean and Flush instruction Enable)를 추가한다. CBCFE=1이면 M보다 낮은 모드에서 `CBO.CLEAN`과 `CBO.FLUSH`를 허용하고, 아니면 불법 명령어 예외를 발생시킨다. Zicbom이 없으면 CBCFE는 읽기 전용 0이다.

Zicbom은 낮은 특권 모드의 `CBO.INVAL` 실행을 제어하는 WARL 필드 CBIE도 추가한다. `00b`이면 불법 명령어 예외가 발생하며 `10b`는 예약이다. Zicbom이 없으면 읽기 전용 0이다. `01b` 또는 `11b`이고 낮은 특권 모드의 실행이 허용되면 다음과 같다.

- `01b`: 실행하되 낮은 모드에서 invalidate를 하도록 설정했어도 flush를 수행한다.
- `11b`: 낮은 모드에서 flush를 하도록 설정하지 않는 한 invalidate를 수행한다.

Smnpm을 구현하면 PMM이 바로 아래 특권 모드의 포인터 마스킹(「포인터 마스킹 확장」)을 표 9에 따라 활성화·비활성화한다. 바로 아래 모드는 S가 있으면 S/HS, 아니면 U이다. Smnpm이 없으면 PMM은 읽기 전용 0이며 RV32에서도 읽기 전용 0이다.

표 9. PMM WARL 필드의 유효 값

값	설명
00	포인터 마스킹 비활성화 PMLEN=0
01	예약됨
10	포인터 마스킹 활성화 PMLEN=XLEN-57 RV64에서 7
11	포인터 마스킹 활성화 PMLEN=XLEN-48 RV64에서 16

Zicfilp는 LPE를 추가한다. LPE=1이고 S모드가 있으면 S모드에서, S모드가 없으면 U모드에서 Zicfilp를 활성화한다. LPE=0이면 S모드에서 활성화하지 않으며 다음 규칙이 적용된다. S모드가 없으면 같은 규칙이 U모드에 적용된다.

- 하트는 ELP를 갱신하지 않고 `NO_LP_EXPECTED`로 유지한다.
- LPAD는 아무 동작도 하지 않는다.

Zicfiss는 SSE를 추가한다. SSE=1이면 S모드에서 Zicfiss를 활성화한다. SSE=0이면 M보다 낮은 특권 모드에 다음 규칙을 적용한다.

- 32비트 Zicfiss 명령어는 Zimop에 정의된 동작으로 돌아간다.
- 16비트 Zicfiss 명령어는 Zcmop에 정의된 동작으로 돌아간다.
- VS/S단계 페이지 테이블의 `pte.xwr=010b`는 예약 인코딩이 된다.
- `SSAMOSWAP.W/D`는 불법 명령어 예외를 발생시킨다.

`menvcfg.SSE=0`이면 `henvcfg.SSE`와 `senvcfg.SSE`는 읽기 전용 0이다.

Ssdbltrp는 DTE(Double Trap Enable)를 추가한다. `menvcfg.DTE=0`이면 Ssdbltrp가 구현되지 않은 것처럼 동작한다. Ssdbltrp가 없으면 `sstatus.SDT`, `vsstatus.SDT`, `henvcfg.DTE`는 읽기 전용 0이다.

XLEN=32이면 `menvcfgh`는 `menvcfg[63:32]`의 별칭인 32비트 읽기/쓰기 레지스터이다. XLEN=64이면 존재하지 않는다. U모드를 지원하지 않으면 `menvcfg`와 `menvcfgh` 모두 존재하지 않는다.

2.1.1.19. 머신 보안 설정 레지스터 mseccfg

`mseccfg`는 그림 25 형식의 64비트 읽기/쓰기 레지스터이며 보안 기능을 제어한다. 이 레지스터에 필드를 추가하는 확장을 하나라도 구현하면 존재하고, 아니면 예약된다.

그림 25. 머신 보안 설정 레지스터 mseccfg

비트 범위	필드
63:34	WPRI
33:32	PMM
31:11	WPRI
10	MLPE
9	SSEED
8	USEED
7:3	WPRI
2	RLB
1	MMWP
0	MML

Zkr은 `mseccfg`에 SSEED와 USEED를 추가하여 M보다 낮은 모드의 `seed` CSR 접근을 제어한다. USEED=0이면 U모드의 `seed` 접근은 불법 명령어 예외를 발생시킨다. USEED=1이면 U모드의 읽기/쓰기 접근을 허용하지만 다른 접근 유형은 불법 명령어 예외를 발생시킨다. Zkr 또는 U모드가 없으면 USEED는 읽기 전용 0이다.

SSEED=0이면 S/HS모드의 `seed` 접근은 불법 명령어 예외를 발생시킨다. SSEED=1이면 읽기/쓰기 접근을 허용하지만 다른 유형은 불법 명령어 예외를 발생시킨다. Zkr 또는 S모드가 없으면 SSEED는 읽기 전용 0이다.

H 확장도 구현한 경우 HS-qualified 명령어를 통한 seed 접근은 VS/VU모드에서 가상 명령어 예외로 이어진다. 그 밖의 접근 유형은 불법 명령어 예외를 발생시킨다.

표 10. 엔트로피 소스 접근 제어

모드	SSEED	USEED	설명
M	—	—	읽기/쓰기 CSR 명령어로 정상 접근 가능. 쓰기 없는 읽기 시도는 모드 및 제어 비트와 관계없이 불법 명령어 예외
U	—	0	모든 seed 접근에서 불법 명령어 예외
U	—	1	정상 접근 가능. 읽기/쓰기는 예외 없음
S/HS	0	—	모든 seed 접근에서 불법 명령어 예외
S/HS	1	—	정상 접근 가능. 읽기/쓰기는 예외 없음
VS/VU	0	—	모든 seed 접근에서 불법 명령어 예외
VS/VU	1	—	읽기/쓰기는 가상 명령어 예외. 그 밖의 접근은 불법 명령어 예외

Smepmp는 RLB, MMWP, MML을 추가한다.

`mseccfg.RLB`(Rule Locking Bypass)는 잠긴 PMP 규칙을 일시적으로 변경할 수 있게 하는 WARL 필드이다. RLB=1이면 잠긴 PMP 규칙을 제거하거나 수정할 수 있다. RLB=0이고 비활성 항목을 포함한 어떤 규칙·항목에서든 `pmpcfg.L`=1이면, RLB는 0으로 유지되며 PMP 리셋 전까지 이후 변경을 모두 무시한다.

> **주석** 이 기능은 디버깅이나 부팅 중 일시적 우회 수단으로, 소프트웨어 단순화 및 메모리·PMP 규칙 배치 최적화를 위해 의도되었다. M모드 전용 규칙의 보호를 약화하므로 부팅 완료 후 정상 동작 중에는 사용을 피하는 것이 바람직하다. 필요 없는 공급업체는 0으로 고정할 수 있다.

Smepmp 필드 정의에 사용하는 용어는「머신 모드의 메모리 접근 및 실행 방지를 위한 PMP 강화 확장 Smepmp」를 참조한다.

`mseccfg.MMWP`(Machine-Mode Allowlist Policy)는 대응 PMP 규칙이 없는 메모리 영역에 M모드가 접근할 때의 기본 정책을 바꾸는 WARL 필드이다. 한 번 설정하면 PMP 리셋까지 지울 수 없는 고정 유지 비트(sticky bit)이다. 설정하면 기본 정책이 PMP 무시에서 접근 거부로 바뀐다.

`mseccfg.MML`(Machine Mode Lockdown)은 2.1.7.1.2절의 `pmpcfg.L` 해석을 바꾸는 WARL 고정 유지 비트이다. 한 번 설정하면 PMP 리셋까지 지울 수 없다. 설정하면 다음과 같이 바뀐다.

1. `pmpcfg.L`은 규칙을 잠그고 모든 모드에 강제한다는 뜻에서, 1이면 M모드 전용이고 0이면 S/U모드 전용이라는 뜻으로 바뀐다. 이전에 예약된 `pmpcfg.RW=01`과 `pmpcfg.LRWX=1111`은 공유 영역(Shared-Region)을 인코딩한다.

M모드 전용 규칙은 M모드에 적용되며 S/U모드 접근은 거부한다. 또한 잠긴 상태이므로 RLB가 1이 아닌 한 PMP 리셋까지 관련 설정 및 주소 레지스터 변경을 무시한다. S/U모드 전용 규칙은 S/U모드에 적용되며 M모드 접근을 거부한다.

공유 영역 규칙은 모든 모드에 적용되되 L과 X에 따라 다음 제약을 갖는다.

- L=0 공유 영역은 M과 S/U 사이 데이터 공유용이므로 실행할 수 없다. M은 읽기/쓰기가 가능하고, S/U는 X=0이면 읽기, X=1이면 읽기/쓰기가 가능하다.
- L=1 공유 영역은 코드 공유용이므로 쓸 수 없다. M과 S/U 모두 실행할 수 있고 X=1이면 M에 읽기도 허용한다. RLB가 1이 아니면 PMP 리셋까지 관련 설정·주소 변경을 무시하는 잠금 상태를 유지한다.
- LRWX=1111은 양쪽 모두 읽기 전용인 데이터 공유 영역을 나타낸다. 이 규칙도 RLB가 1이 아니면 PMP 리셋까지 잠긴다.

2. M모드 전용이거나 잠긴 공유 영역인 실행 가능 규칙을 추가할 수 없다. 해당 pmpcfg 쓰기는 무시하고 기존 값을 유지한다. 부팅 중 등에서 RLB=1로 이 제약을 일시 해제할 수 있다.

3. M모드 권한으로 코드를 실행하려면 실행 권한이 있는 M모드 전용 또는 잠긴 공유 영역 규칙과 일치해야 한다. 대응 규칙이 없거나 S/U 전용 규칙과 일치하는 영역의 실행은 거부한다.

4. MML이 0이면 pmpcfg.RW=01 조합은 향후 표준용 예약으로 유지된다.

Smmpm을 구현하면 PMM이 표 11에 따라 M모드의 포인터 마스킹(「포인터 마스킹 확장」)을 활성화·비활성화한다. Smmpm이 없거나 RV32이면 PMM은 읽기 전용 0이다.

표 11. PMM WARL 필드의 유효 값

값	설명
00	포인터 마스킹 비활성화 PMLEN=0
01	예약됨
10	포인터 마스킹 활성화 PMLEN=XLEN-57 RV64에서 7
11	포인터 마스킹 활성화 PMLEN=XLEN-48 RV64에서 16

주석 Smmpm 구현은 TLB 접근 경로에 마스킹 논리를 두지 않으려면 max(지원 최대 가상 주소 폭, 지원 최대 슈퍼바이저 물리 주소 폭) ≤ XLEN-PMLEN비트를 만족해야 한다.

Zicfilp는 MLPE를 추가한다. MLPE=1이면 M모드에서 Zicfilp를 활성화한다. MLPE=0이면 활성화하지 않으며 M모드에 다음 규칙을 적용한다.

- 하트는 ELP를 갱신하지 않고 NO_LP_EXPECTED로 유지한다.
- LPAD는 아무 동작도 하지 않는다.

XLEN=32일 때만 mseccfgh가 mseccfg[63:32]의 별칭인 32비트 읽기/쓰기 레지스터이다. XLEN=32이고 mseccfg가 구현되면 존재하며 XLEN=64에서는 존재하지 않는다.

2.1.2. 머신 수준 메모리 매핑 레지스터

2.1.2.1. 머신 타이머 레지스터 mtime 및 mtimecmp

플랫폼은 메모리 매핑 M모드 읽기/쓰기 레지스터 mtime으로 실시간 카운터를 제공한다. mtime은 일정한 주파수로 증가해야 하며 플랫폼은 한 틱의 주기를 알아내는 방법을 제공해야 한다. 카운트가 오버플로되면 처음으로 되돌아간다.

모든 RV32 및 RV64 시스템에서 mtime은 64비트 정밀도이다. 플랫폼은 64비트 메모리 매핑 M모드 타이머 비교 레지스터 mtimecmp도 제공한다. 두 값을 부호 없는 정수로 취급하여 mtime≥mtimecmp이면 머신 타이머 인터럽트가 대기 상태가 된다. 보통 mtimecmp 쓰기로 mtimecmp>mtime이 될 때까지 요청은 유지된다. 실제 인터럽트 수신은 인터럽트가 활성화되어 있고 mie.MTIE=1일 때만 일어난다.

그림 26. 머신 시간 레지스터 메모리 매핑 제어 레지스터

비트 범위	필드
63:0	mtime

그림 27. 머신 시간 비교 레지스터 메모리 매핑 제어 레지스터

비트 범위	필드
63:0	mtimecmp

주석 타이머는 사이클 카운터 대신 실제 경과 시간을 사용하도록 정의한다. 이는 동적 전압·주파수 조정으로 에너지를 절약하는 현대 프로세서의 가변 클록 주파수를 지원하기 위함이다. 정확한 RTC는 수정 또는 MEMS 발진기가 필요해 비교적 비싸고 시스템 나머지 부분의 전원이 꺼져도 동작해야 한다. 그래서 보통 시스템에 하나만 있으며 프로세서와 다른 주파수·전압 도메인에 있다. 모든 하트가 공유해야 하므로 접근에는 전압 레벨 변환과 클록 도메인 교차 비용이 생길 수 있다. 따라서 CSR보다 메모리 매핑 레지스터로 노출하는 것이 자연스럽다. 낮은 특권 수준에는 자체 `timecmp`가 없다. 대신 M 모드 소프트웨어가 다음 타이머 인터럽트를 `mtimecmp`에 다중화하여 하트마다 임의 개수의 가상 타이머를 구현할 수 있다. 단순한 고정 주파수 시스템은 사이클 계수와 실제 시간에 같은 클록을 사용할 수 있다.

`mtime`과 `mtimecmp`의 비교 결과가 바뀌면 즉시는 아닐 수 있지만 결국 MTIP에 반영되어야 한다.

주석 처리기가 `mtimecmp`를 증가시킨 뒤 즉시 복귀하면 아직 MTIP가 내려가지 않아 불필요한 타이머 인터럽트가 한 번 발생할 수 있다. 모든 소프트웨어는 이 가능성을 고려해야 하지만, 대부분은 매우 드문 사건으로 가정해도 된다. MTIP가 내려갈 때까지 폴링하는 것보다 가끔 발생하는 불필요한 인터럽트를 감수하는 것이 거의 항상 더 빠르다.

RV32에서는 `mtimecmp`에 대한 메모리 매핑 쓰기가 32비트 절반 하나만 바꾼다. 아래 순서는 비교 값의 중간 상태로 불필요한 인터럽트가 생기지 않게 64비트 값을 설정한다. RV64에서는 추가로 `mtime`과 `mtimecmp`에 대한 자연 정렬된 64비트 메모리 접근을 지원하며 이는 원자적이다.

다음은 리틀 엔디언이고 레지스터가 강하게 순서화된 I/O 영역에 있다고 가정한 RV32 예이다. 하위 절반에 -1을 먼저 쓰면 설정 중 값이 기존 값과 새 값 중 작은 값보다 작아지는 것을 막는다.

```
# 새 비교 값은 a1:a0에 있다.
li t0, -1
la t1, mtimecmp
sw t0, 0(t1)     # 기존 값보다 작아지지 않음.
sw a1, 4(t1)     # 새 값보다 작아지지 않음.
sw a0, 0(t1)     # 새 값.
```

`time` CSR은 메모리 매핑 `mtime`의 읽기 전용 사본이다. XLEN=32이면 `timeh`가 상위 32비트를, `time`이 하위 32비트만 나타낸다. `mtime` 변경은 즉시가 아닐 수 있지만 결국 `time`과 `timeh`에 반영되어야 한다.

2.1.3. 머신 모드 특권 명령어

2.1.3.1. 환경 호출 및 중단점

명령어	31:20	19:15	14:12	11:7	6:0
ECALL	000000000000	00000	000	00000	1110011
EBREAK	000000000001	00000	000	00000	1110011

ECALL은 지원 실행 환경에 요청하기 위한 명령어이다. U, S, M모드에서 실행하면 각각 U모드·S모드·M모드 환경 호출 예외를 발생시키며 다른 동작은 하지 않는다.

주석 호출한 특권 모드마다 다른 예외를 발생시키므로 선택적으로 위임할 수 있다. Unix 계열 OS는 보통 U모드 환경 호출만 S모드에 위임하고 나머지는 위임하지 않는다.

EBREAK는 디버거가 제어를 디버깅 환경으로 돌려보내는 데 사용한다. 외부 디버그 환경이 별도로 동작을 바꾸지 않는 한 중단점 예외를 발생시키며 다른 동작은 하지 않는다.

주석 제I권「압축 명령어 표준 확장 C」의 C.EBREAK는 EBREAK와 같은 동작을 한다.

ECALL과 EBREAK는 트랩을 받는 모드의 epc를 다음 명령어가 아니라 ECALL/EBREAK 자신의 주소로 설정한다. 동기 예외를 일으키므로 완료한 명령어로 간주하지 않으며 minstret를 증가시키지 않는 것이 바람직하다.

2.1.3.2. 트랩 복귀 명령어

트랩 복귀 명령어는 PRIV 부 연산 코드에 인코딩된다.

명령어	31:20	19:15	14:12	11:7	6:0
MRET	001100000010	00000	000	00000	1110011
SRET	000100000010	00000	000	00000	1110011

특권 수준별로 MRET와 SRET가 제공된다. MRET는 항상 제공한다. S모드를 지원하면 SRET를 제공해야 하고, 아니면 불법 명령어 예외를 발생시키는 것이 바람직하다. 2.1.1.6.6절과 같이 mstatus.TSR=1일 때도 SRET는 불법 명령어 예외를 발생시키는 것이 바람직하다.

xRET는 모드 x 또는 그보다 높은 모드에서 실행할 수 있다. 낮은 수준의 xRET를 실행하면 해당 낮은 수준의 인터럽트 활성화·특권 모드 스택을 꺼낸다. x보다 낮은 모드에서 실행하려 하면 불법 명령어 예외가 발생한다. 2.1.1.6.1절의 특권 스택 조작 외에도 pc를 xepc 값으로 설정한다.

A 확장을 지원하면 xRET가 남아 있는 LR 주소 예약을 지울 수 있으나 의무는 아니다. 예약 해제가 필요하면 트랩 처리기는 xRET 전에 더미 SC 등을 사용하여 명시적으로 지우는 것이 바람직하다.

주석 xRET가 항상 LR 예약을 지우면 디버거로 LR/SC 순서열을 한 명령어씩 실행할 수 없게 된다.

2.1.3.3. 인터럽트 대기

WFI(Wait for Interrupt)는 인터럽트 처리가 필요해질 때까지 현재 하트를 정지시킬 수 있음을 구현체에 알린다. 또한 적절한 인터럽트를 이 하트에 우선 전달하라는 뜻으로 플랫폼에 알릴 수 있다. 모든 특권 모드에서 제공하며 U 모드 지원은 선택 사항이다. 2.1.1.6.6절과 같이 mstatus.TW=1이면 불법 명령어 예외가 발생할 수 있다.

명령어	31:20	19:15	14:12	11:7	6:0
WFI	000100000101	00000	000	00000	1110011

하트가 정지한 동안 활성화된 인터럽트가 이미 있거나 새로 생기면, WFI 다음 명령어에서 인터럽트 트랩을 수신한다. 즉 처리기에서 실행을 재개하고 mepc=pc+4가 된다.

주석 다음 명령어에서 트랩하도록 한 것은 단순히 처리기에서 복귀하면 WFI 다음 코드를 실행하게 하기 위해서이다.

구현체는 활성화된 인터럽트가 대기하지 않더라도 어떤 이유로든 실행을 재개할 수 있다. 따라서 WFI를 NOP로 구현하는 것도 유효하다.

주석 WFI에서 하트를 정지시키지 않으면 인터럽트는 WFI를 포함한 유휴 루프의 어떤 명령어에서 수신된다. 처리기에서 단순 복귀하면 유휴 루프 실행이 재개된다.

인터럽트가 비활성화되어 있어도 WFI를 실행할 수 있다. WFI 동작은 mstatus의 전역 MIE·SIE와 mideleg의 영향을 받아서는 안 된다. 즉 개별적으로 활성화된 인터럽트가 대기하면, 더 낮은 모드에 위임되었더라도 재개해야 한다. 다만 MTIE 같은 개별 활성화 비트는 따르는 것이 바람직하다. 대기 중이어도 개별 활성화되지 않은 인터럽트 때문에 재개하는 것은 피하는 것이 바람직하다. 어느 특권 수준의 인터럽트이든 개별적으로 활성화되어 대기하면, 해당 수준의 전역 활성화 값과 관계없이 WFI에서 재개해야 한다.

재개 원인 사건이 실제 인터럽트 수신으로 이어지지 않으면 pc+4에서 재개한다. 소프트웨어는 조치할 사건이 없을 때 WFI로 되돌아가는 것 등을 포함하여 다음 동작을 결정해야 한다.

주석 인터럽트가 꺼져 있어도 깨어날 수 있으면 WFI 전에 현재 문맥을 저장하거나 폐기한 뒤, 문맥 저장이 필요 없는 별도의 처리기 진입점을 호출할 수 있다. 구현은 WFI를 NOP로 만들 수 있으므로 후속 코드에서 관련된 대기·비활성 인터럽트를 명시적으로 검사해야 하며, 적절한 인터럽트가 없으면 WFI로 되돌아가는 것이 바람직하다.

M모드에서는 mip, S모드에서는 sip를 조사하여 인터럽트 존재를 확인할 수 있다. WFI는 위임 레지스터 설정의 영향을 받지 않는다. 구현체는 WFI를 만난 즉시 또는 일정 시간 뒤 높은 모드로 트랩하여, 예를 들어 M모드에서 저전력 상태 전환을 시작할 수 있다. 같은 '사건 대기' 형태는 향후 메모리 위치 변경이나 메시지 도착을 기다리는 확장에도 사용할 수 있다.

2.1.3.4. 사용자 정의 SYSTEM 명령어

그림 28의 SYSTEM 주 연산 코드 하위 공간은 사용자 정의용이다. 다른 SYSTEM 명령어처럼 비트 29:28로 최소 필요 특권 모드를 나타내는 것을 권장한다.

그림 28. 사용자 정의용 SYSTEM 명령어 인코딩

31:26 funct6	25:15	14:12	11:7	6:0	권장 용도
100011	사용자 정의	000	사용자 정의	1110011	비특권 또는 U수준
110011	사용자 정의	000	사용자 정의	1110011	비특권 또는 U수준
100111	사용자 정의	000	사용자 정의	1110011	S수준
110111	사용자 정의	000	사용자 정의	1110011	S수준
101011	사용자 정의	000	사용자 정의	1110011	HS수준
111011	사용자 정의	000	사용자 정의	1110011	HS수준
101111	사용자 정의	000	사용자 정의	1110011	M수준
111111	사용자 정의	000	사용자 정의	1110011	M수준

2.1.4. 리셋

리셋 시 하트는 M모드가 되고 mstatus.MIE와 MPRV는 0이 된다. 리틀 엔디언 메모리 접근을 지원하면 mstatus/mstatush의 MBE도 0이 된다. misa는 2.1.1.1절과 같이 지원 확장의 최대 집합을 활성화한다. A 확장 구현에는 유효한 LR 예약이 남아 있지 않는다. pc는 구현 정의 리셋 벡터가 된다.

mcause는 리셋 원인 값으로 설정한다. 쓰기 가능한 PMP의 A와 L은 플랫폼이 다른 리셋 값을 요구하지 않는 한 0이다. H 확장이 있으면 hgatp.MODE와 vsatp.MODE는 0이다. Smrnmi가 있으면 mnstatus.NMIE는 0이다. 어떤 WARL 필드도 유효하지 않은 값을 포함하지 않는다. Zicfilp가 있으면 mseccfg.MLPE는 0이다. 그 밖의 모든 하트 상태는 UNSPECIFIED이다. mseccfg의 MML, MMWP, RLB는 플랫폼이 다른 값을 요구하지 않으면 0이다.

리셋 후 mcause 해석은 구현별이지만, 리셋 조건을 구분하지 않는 구현은 0을 반환하는 것이 바람직하다. 구분하는 구현은 가장 완전한 리셋에만 0을 사용하는 것이 바람직하다. mseccfg.USEED와 SSEED는 정의된 리셋 값을 가져야 하며, 리셋 후 정의되지 않은 상태를 허용해서는 안 된다.

주석 전원 인가, 외부 하드 리셋, 전압 저하 검출, 워치독 만료, 절전 모드 기상 등 여러 리셋 원인을 M모드 소프트웨어와 디버거가 구분하고 싶을 수 있다. 리셋 `mcause` 값은 동기 예외 값과 겹칠 수 있다. 리셋 시 `pc`는 보통 다른 트랩과 다른 값이므로 이 중복은 모호하지 않다.

2.1.5. 마스킹 불가능 인터럽트

NMI는 하드웨어 오류 상황에만 사용하며, 인터럽트 활성화 비트와 관계없이 구현 정의의 M모드 NMI 벡터로 즉시 점프한다. `mepc`에는 중단된 명령어 가상 주소를, `mcause`에는 NMI 원인을 기록한다. 따라서 실행 중인 M모드 인터럽트 처리기의 상태를 덮어쓸 수 있다.

NMI의 `mcause` 값은 구현 정의이다. 최상위 Interrupt 비트는 인터럽트임을 나타내도록 설정하는 것이 바람직하다. Exception Code=0은 '알 수 없는 원인'으로 예약된다. NMI 소스를 구분하지 않는 구현은 이 코드에 0을 반환하는 것이 바람직하다.

NMI는 리셋과 달리 프로세서 상태를 초기화하지 않으므로 하드웨어 오류의 진단·보고 및 가능한 영향 제한이 가능하다.

2.1.6. 물리 메모리 속성

전체 시스템의 물리 메모리 맵에는 일반 메모리 및 메모리 매핑 제어 레지스터 등 다양한 주소 범위가 있으며 일부에는 접근할 수 없을 수 있다. 읽기·쓰기·실행, 워드/블록보다 작은 단위의 접근, 원자적 연산, 캐시 일관성을 지원하지 않는 영역이 있거나 메모리 모델이 다를 수 있다. 제어 레지스터도 접근 폭, 원자적 연산, 읽기·쓰기 부수 효과가 서로 다르다. RISC-V에서는 물리 주소 공간 각 영역의 이러한 속성과 기능을 **물리 메모리 속성(PMA)**이라고 한다. 이 절은 용어 및 구현·검사 방법을 설명한다.

PMA는 기반 하드웨어 고유 특성이며 동작 중에는 거의 바뀌지 않는다. 2.1.7절의 PMP와 달리 실행 문맥별로 달라지지 않는다. 온칩 ROM처럼 칩 설계 때 정하는 것도 있고, 외부 버스에 어떤 칩을 연결하는지 등에 따라 보드 설계 때 정하는 것도 있다. 외부 버스 장치는 전원을 껐다 켤 때 교체하는 콜드 플러그나 동작 중 교체하는 핫 플러그를 지원할 수 있다. 일부 장치는 실행 중 다른 PMA의 용도로 설정할 수 있다. 예를 들어 온칩 스크래치패드 RAM을 한 응용에서는 코어 하나의 전용 캐시 대상으로, 다른 응용에서는 공유 비캐시 메모리로 사용할 수 있다.

모든 물리 주소에서 지원되지 않는 연산이 있고 설정 가능한 PMA의 현재 값을 알아야 하는 연산도 있으므로, 대부분의 시스템에서는 물리 주소가 확인된 파이프라인 후반에 일부 PMA를 하드웨어로 동적 검사해야 한다. 다른 아키텍처처럼 일부 PMA를 가상 메모리 페이지 테이블에 넣고 TLB로 파이프라인에 전달하면 플랫폼별 정보를 가상화 계층에 넣게 된다. 각 물리 영역에 대한 모든 PTE 속성을 올바르게 초기화하지 않으면 시스템 오류가 날 수 있다. 또한 지원 페이지 크기가 물리 속성 지정에 적합하지 않아 주소 공간 파편화와 비싼 TLB 항목의 낭비를 일으킬 수 있다.

RISC-V는 PMA의 명세와 검사를 별도 하드웨어 PMA 검사기로 분리한다. 영역별 속성을 설계 때 알면 상수로 구현할 수 있다. 실행 중 설정 가능하면 각 영역에 적절한 단위의 플랫폼별 메모리 매핑 제어 레지스터로 지정할 수 있다. 예를 들어 온칩 SRAM을 캐시 가능·불가능 용도로 유연하게 분할할 수 있다. 가상→물리 변환을 거친 접근을 포함한 모든 물리 메모리 접근에서 PMA를 검사한다.

시스템 디버깅을 돕기 위해 가능한 한 PMA 검사 실패 접근에서 정확한 트랩을 발생시키는 것을 강하게 권장한다. 이러한 위반은 가상 메모리 페이지 폴트와 구분되는 명령어·로드·스토어 접근 폴트 예외로 나타난다. 다만 접근 실패를 탐색에 사용하는 구형 버스 조사 등의 경우 항상 정확한 트랩이 가능한 것은 아니다. 이때 주변 장치의 오류 응답은 부정확한 버스 오류 인터럽트로 보고한다.

특정 장치에 올바르게 접근하거나 DMA 엔진 등 메모리 접근 하드웨어를 설정하려면 소프트웨어도 PMA를 읽을 수 있어야 한다. PMA는 플랫폼 구성에 밀접하여 세부 내용과 확인 방법이 본질적으로 플랫폼별이다. 특히 구형 버스 등 일부 장치는 PMA 탐색을 지원하지 않아 미지원 접근 시 오류 응답이나 타임아웃을 낸다. 보통 플랫폼별 M모드 코드가 PMA를 추출하여 어떤 표준 표현으로 상위의 낮은 특권 소프트웨어에 제공한다.

동적 PMA 재설정을 지원하는 플랫폼은 M모드 드라이버에 요청하여 속성을 바꾸는 인터페이스를 제공한다. 예를 들어 캐시 가능 속성 변경에는 M모드에서만 가능한 플랫폼별 캐시 플러시 등이 필요할 수 있다.

2.1.6.1. 주 메모리와 I/O 영역

주소 범위의 가장 중요한 구분은 일반 주 메모리인지 I/O 장치인지이다. 일반 주 메모리는 아래의 여러 특성을 충족해야 하지만 I/O 속성은 훨씬 다양할 수 있다. 장치 스크래치패드 RAM 등 일반 주 메모리에 해당하지 않는 영역은 I/O로 분류한다.

> **주석** 이전 명세의 빈 영역(vacant region)은 독립 범주가 아니며, 읽기·쓰기·실행 권한이 없는 접근 불가 I/O 영역으로 설명한다. 접근 불가 주 메모리 영역도 허용한다.

2.1.6.2. 지원 접근 유형 PMA

접근 유형은 8비트 바이트부터 긴 다중 워드 버스트까지 지원 접근 폭을 정하고, 각 폭에서 비정렬 접근을 지원하는지도 정한다.

> **주석** 하트의 소프트웨어는 메모리 버스트를 직접 만들 수 없지만 DMA 엔진에 I/O 장치 접근을 설정할 수 있으므로 지원 접근 크기를 알아야 할 수 있다.

주 메모리는 연결 장치가 요구하는 모든 접근 폭의 읽기·쓰기를 항상 지원하며 명령어 인출 지원 여부를 지정할 수 있다.

> **주석** 플랫폼에 따라 주 메모리 전체의 명령어 인출을 요구하거나 일부 영역에서는 금지할 수 있다. 프로세서나 장치가 다른 접근 폭도 지원하는 경우가 있어도 주 메모리가 지원하는 유형만으로 동작할 수 있어야 한다.

I/O 영역은 어떤 데이터 폭에 대해 어떤 읽기·쓰기·실행 조합을 지원하는지 지정할 수 있다. 페이지 기반 가상 메모리 시스템의 I/O 및 메모리 영역은 하드웨어 페이지 테이블 읽기·쓰기의 지원 조합을 지정할 수 있다.

> **주석** Unix 계열 OS는 보통 캐시 가능한 주 메모리 전체가 페이지 테이블 순회를 지원하도록 요구한다.

2.1.6.3. 원자성 PMA

원자성 PMA는 주소 영역이 지원하는 원자적 명령어를 설명한다. 지원은 LR/SC와 AMO의 두 부류로 나뉜다.

> **주석** 일부 플랫폼은 캐시 가능한 주 메모리 전체가 연결된 프로세서에 필요한 모든 원자적 연산을 지원하도록 요구할 수 있다.

2.1.6.3.1. AMO PMA

AMO 지원 수준은 AMONone, AMOSwap, AMOLogical, AMOArithmetic의 네 가지이다. AMONone은 AMO 미지원, AMOSwap은 해당 범위의 `amoswap`만 지원, AMOLogical은 교환 및 모든 논리 AMO(`amoand`, `amoor`, `amoxor`) 지원, AMOArithmetic은 A 확장의 모든 AMO 지원을 뜻한다. 각 수준에서 기반 메모리가 특정 폭의 읽기·쓰기를 지원하면 그 폭의 자연 정렬 AMO도 지원한다. 주 메모리와 I/O는 프로세서가 지원하는 원자적 연산 중 일부만 지원하거나 전혀 지원하지 않을 수 있다.

표 12. I/O 영역이 지원하는 AMO 부류

AMO 부류	지원 연산
AMONone	없음
AMOSwap	amoswap
AMOLogical	위 연산 + amoand, amoor, amoxor
AMOArithmetic	위 연산 + amoadd, amomin, amomax, amominu, amomaxu

주석 가능한 경우 I/O 영역에서 적어도 AMOLogical을 지원하는 것을 권장한다.

Zacas는 AMOCASW, AMOCASD, AMOCASQ의 세 수준을 추가한다. AMOCASW는 AMOArithmetic에 `AMOCAS.W`를, AMOCASD는 AMOCASW에 `AMOCAS.D`를, AMOCASQ는 AMOCASD에 `AMOCAS.Q`를 추가로 지원한다.

주석 `AMOCAS.W/D/Q`는 산술 비교 및 교환 능력이 필요하므로 AMOCASW/D/Q에는 AMOArithmetic 지원이 필요하다.

Zabha의 AMO는 A 또는 Zacas의 대응 명령어와 같은 지원 수준을 요구한다.

2.1.6.3.2. 예약 가능성 PMA

LR/SC에는 예약 가능성과 최종 성공 보장의 조합을 나타내는 RsrvNone, RsrvNonEventual, RsrvEventual의 세 수준이 있다. RsrvNone은 LR/SC를 지원하지 않아 해당 위치에 예약할 수 없음을 뜻한다. RsrvNonEventual은 지원하여 예약은 가능하지만 비특권 ISA의 최종 성공 보장은 제공하지 않는다. RsrvEventual은 지원하며 최종 성공도 보장한다.

주석 가능한 경우 주 메모리의 RsrvEventual 지원을 권장한다. LR/SC는 캐시 일관성 위에 구현하는 것이 편리하므로 대부분의 I/O 영역은 지원하지 않을 것이다. 다만 일부는 RsrvNonEventual 또는 RsrvEventual을 지원할 수 있다. RsrvNonEventual 위치에서 LR/SC를 사용하면 진행이 없음을 감지했을 때 사용할 대체 수단을 소프트웨어가 제공하는 것이 바람직하다.

2.1.6.4. 비정렬 원자성 단위 PMA

비정렬 원자성 단위(misaligned atomicity granule) PMA는 비정렬 AMO를 제한적으로 지원한다. 존재하면 자연 정렬된 2의 거듭제곱 바이트 수인 단위 크기를 지정한다. 지원 값은 MAGNN으로 표시한다. 예를 들어 MAG16은 단위가 적어도 16바이트임을 뜻한다.

이 PMA는 AMO, 기본 ISA의 로드·스토어, F/D/Q의 XLEN비트 이하 로드·스토어 및 이들의 압축 인코딩에만 적용된다. 이 집합의 명령어가 접근하는 모든 바이트가 같은 단위 안에 있으면 주소 정렬을 이유로 예외를 발생시키지 않으며 RVWMO 관점에서 하나의 메모리 연산만 생성한다. 즉 원자적으로 실행된다.

비정렬 AMO가 이 PMA 없는 영역에 접근하거나 모든 바이트가 같은 단위 안에 있지 않으면 예외가 발생한다. 일반 로드·스토어에서는 예외가 발생하거나 접근은 진행하되 원자성은 보장하지 않는다. 일부 비정렬 접근에는 주소 비정렬 대신 접근 폴트를 발생시켜 처리기가 해당 명령어를 에뮬레이션해서는 안 됨을 나타낼 수 있다.

주석 LR/SC는 이 PMA의 영향을 받지 않으며 비정렬이면 항상 예외를 발생시킨다. 벡터 메모리 접근도 영향을 받지 않으므로 같은 단위 안에 있어도 비원자적으로 실행될 수 있다. 암묵적 접근도 영향을 받지 않는다.

2.1.6.5. 메모리 순서 PMA

FENCE 및 원자적 명령어 순서 비트의 관점에서 주소 공간은 주 메모리와 I/O로 분류한다. 한 하트의 주 메모리 접근은 다른 하트뿐 아니라 DMA처럼 주 메모리 요청을 시작할 수 있는 장치에도 보인다. 일관된 주 메모리는 항상 RVWMO 또는 RVTSO 모델을 사용한다. 일관되지 않은 주 메모리의 모델은 구현 정의이다.

I/O 접근은 다른 하트와 버스 마스터 장치뿐 아니라 대상 I/O 장치에도 보인다. I/O 영역은 완화된 순서(relaxed ordering) 또는 강한 순서(strong ordering)로 접근할 수 있다. 완화된 I/O 접근은 제I권「RVWMO 설명 자료」부록의 I/O 순서 절과 같이 RVWMO 메모리 접근과 유사한 순서로 관찰된다. 강하게 순서화된 I/O 접근은 일반적으로 프로그램 순서대로 관찰된다.

각 강하게 순서화된 I/O 영역은 영역 간 순서 보장 수단인 번호가 붙은 순서 채널을 지정한다. 채널 0은 점대점 강한 순서만 나타내며 하트의 해당 단일 I/O 영역 접근끼리만 강하게 순서화한다. 채널 1은 모든 I/O 영역에 걸친 전역 강한 순서를 제공한다. 하트의 채널 1 영역 접근은, 같은 하트의 완화된 영역이나 다른 번호의 강한 순서 영역 접근과의 관계까지 포함해, 모든 다른 하트 및 I/O 장치에 프로그램 순서로만 보인다. 즉 채널 1 접근은 그 명령어 앞뒤에 `fence io,io`를 실행하는 것과 같다. 더 큰 채널 번호는 같은 번호의 모든 영역에 대한 해당 하트 접근을 프로그램 순서로 보장한다. 시스템은 영역별 순서 속성의 동적 설정을 지원할 수 있다.

> **주석** 강한 순서는 구형 장치 드라이버와의 호환성을 높이거나, 구현이 재정렬하지 않음을 알 때 명시적 순서 명령어보다 성능을 높일 수 있다. 하트와 장치 사이에 순서를 유지하는 통신 경로가 하나면 제공하기 쉬우므로 로컬 강한 순서인 채널 0이 기본이다. 같은 상호 연결 경로를 공유하고 요청을 재정렬하지 않는 서로 다른 강한 순서 I/O 영역은 추가 하드웨어 없이 같은 채널을 공유할 수 있다.

2.1.6.6. 일관성 및 캐시 가능성 PMA

일관성(coherence)은 하나의 물리 주소에 대해 정의되며 한 에이전트의 쓰기가 결국 다른 일관성 참여 에이전트에 보임을 뜻한다. 전체 메모리의 읽기·쓰기 이력에서 어떤 값을 읽을 수 있는지 정의하는 메모리 일관성 모델(memory consistency model)과 혼동해서는 안 된다. RISC-V에서는 소프트웨어 복잡도, 성능 및 에너지 영향 때문에 하드웨어 비일관 영역 사용을 권장하지 않는다.

캐시 가능성은 주 메모리/I/O 분류, 순서, 지원 접근·원자적 연산, 일관성 등 다른 PMA에 반영된 차이 외에는 소프트웨어가 보는 동작에 영향을 주지 않는 것이 바람직하다. 따라서 M모드 소프트웨어만 관리하는 플랫폼 수준 설정으로 취급한다. 설정 가능하면 플랫폼별 M모드 루틴이 값을 바꾸고 필요 시 캐시를 비워, 전환 동안에만 비일관 상태가 되도록 한다. 이 일시 상태는 낮은 특권 수준에 보이지 않는 것이 바람직하다.

> **주석** 어느 에이전트도 캐시하지 않는 공유 영역은 일관성을 쉽게 제공할 수 있다. PMA는 사설·공유 캐시에 저장하면 안 됨을 나타내면 된다. 읽기 전용 영역도 여러 에이전트가 일관성 프로토콜 없이 안전하게 캐시할 수 있으며, PMA는 캐시 가능하나 쓰기 불가라고 나타낸다. 읽기/쓰기 영역이 한 에이전트에만 접근 가능하면 일관성 프로토콜 없이 그 에이전트의 사설 캐시나 공유 캐시에 둘 수 있다. 다른 에이전트도 접근할 수 있는 읽기/쓰기 영역을 캐시하면, 다른 에이전트의 캐시 여부와 관계없이 오래된 값 사용을 막기 위한 캐시 일관성 메커니즘이 필요하다.
>
> 하드웨어 비일관 영역의 일관성은 전부 소프트웨어로 구현할 수 있으나 올바르게 만들기 매우 어렵고 보수적인 캐시 플러시로 성능을 크게 해칠 수 있다. 하드웨어 방식은 회로가 복잡하고 일관성 탐색으로 성능에 영향을 줄 수 있으나 그 밖에는 소프트웨어에 보이지 않는다. 하드웨어 일관 영역의 PMA는 일관됨과, 제어기가 여러 개면 어느 것을 사용할지 나타낸다. 외부 수준 공유 캐시가 제어기이고 더 바깥의 제어기에 계층적으로 접근하는 시스템도 있다.
>
> 대부분의 영역은 비캐시, 읽기 전용, 하드웨어 캐시 일관 또는 단일 에이전트 전용 중 하나이므로 소프트웨어 관점에서 일관적이다.

PMA가 캐시 불가를 나타내면 접근은 어떤 캐시도 아닌 메모리 자체에서 충족해야 한다.

> **주석** 캐시 가능성 제어를 지원하면 현재 캐시에 있는 위치에 프로그램이 비캐시 접근할 수 있다. 이때 캐시 복사본을 무시해야 한다. 높은 특권 모드의 추측적 캐시 재충전이 낮은 모드의 비캐시 접근 동작에 영향을 주지 않도록 하기 위한 제약이다.

2.1.6.7. 멱 등성 PMA

멱 등성(idempotency) PMA는 주소 영역의 읽기·쓰기가 멱 등적인지 설명한다. 주 메모리는 멱 등적이라고 가정한다. I/O는 읽기와 쓰기를 별도로 지정할 수 있다. 예를 들어 읽기는 멱 등적이지만 쓰기는 아닐 수 있다. 비멱 등 접근, 즉 읽기나 쓰기마다 부수 효과가 있을 수 있는 경우 추측적 또는 중복 접근을 피해야 한다. 중복 접근이 관찰되는 메모리 순서를 바꾸는 것은 이 PMA 정의의 부수 효과로 간주하지 않는다.

> **주석** 비멱 등 영역의 추측·중복 접근을 피하도록 하드웨어를 설계해야 할 뿐 아니라, 소프트웨어나 컴파일러 최적화도 불필요한 접근을 만들지 않도록 해야 한다. 비멱 등 영역은 비정렬 접근을 지원하지 않을 수 있다. 그런 접근에는 주소 비정렬보다 접근 폴트를 발생시키는 것이 바람직하다. 여러 작은 접근으로 에뮬레이션하면 예상치 못한 부수 효과가 생길 수 있으므로 그렇게 해서는 안 됨을 나타내기 위해서이다.

비멱 등 영역의 암묵적 읽기·쓰기는 앞당기거나 추측적으로 수행해서는 안 된다. 단, 비추측적 암묵 읽기를 수행할 때는 그 주소를 포함하는 자연 정렬된 2의 거듭제곱 크기 영역 안의 다른 바이트도 추가로 읽을 수 있다. 비추측적 명령어 인출에서는 같은 크기의 바로 다음 자연 정렬 영역도 추가로 읽을 수 있으며, 영역 주소는 2^{XLEN}을 법으로 계산한다. 추가 읽기의 결과를 후속 선행·추측 암묵 읽기에 사용할 수 있다. 영역 크기는 구현 정의이나 페이지 기반 가상 메모리 시스템에서는 지원하는 최소 페이지보다 클 수 없다.

2.1.7. 물리 메모리 보호

안전한 처리와 오류 확산 제한을 위해 소프트웨어가 접근 가능한 물리 주소를 제한하는 것이 바람직하다. 선택적인 물리 메모리 보호(PMP) 장치는 하트별 M모드 제어 레지스터로 영역별 읽기·쓰기·실행 권한을 지정하게 한다. PMP 검사는 2.1.6절의 PMA 검사와 병렬로 수행한다.

접근 제어 설정 단위는 플랫폼별이지만 표준 PMP 인코딩은 최소 4바이트 영역을 지원한다. 특정 영역 권한은 상수로 고정할 수 있다. 예를 들어 M모드에서만 보이고 낮은 계층에서는 절대 보이지 않는 영역을 만들 수 있다.

> **주석** 플랫폼마다 물리 보호 요구가 크게 다르므로 이 절의 방식에 추가하거나 대신하여 다른 PMP 구조를 제공할 수 있다.

PMP는 유효 특권 모드가 S 또는 U인 모든 접근에 적용된다. S/U의 명령어 인출·데이터 접근, `mstatus.MPRV=1`이고 MPP가 S/U인 M모드 데이터 접근이 포함된다. 가상 주소 변환의 페이지 테이블 접근에도 적용하며 그 유효 특권 모드는 S이다. 선택적으로 M모드 접근에도 적용할 수 있으며 이때 PMP 레지스터를 잠가 하트 리셋 전에는 M모드 소프트웨어도 바꾸지 못하게 한다. 즉 기본적으로 권한이 없는 S/U에는 권한을 줄 수 있고, 기본적으로 모든 권한이 있는 M에서는 권한을 회수할 수 있다. PMP 위반은 프로세서에서 항상 정확한 트랩으로 처리한다.

2.1.7.1. 물리 메모리 보호 CSR

각 PMP 항목은 8비트 설정 레지스터와 MXLEN비트 주소 레지스터 하나로 설명한다. 일부 설정은 바로 앞 항목의 주소 레지스터도 사용한다. 최대 64개를 지원하며 구현은 0, 16, 64개 중 하나를 선택할 수 있다. 번호가 낮은 항목부터 먼저 구현해야 한다. 모든 PMP CSR 필드는 WARL이며 읽기 전용 0일 수 있다. PMP CSR은 M모드에서만 접근한다.

문맥 교환 시간을 줄이려고 설정 레지스터를 CSR 안에 조밀하게 묶는다. RV32는 그림 29처럼 `pmpcfg0~pmpcfg15`의 16개 CSR에 `pmp0cfg~pmp63cfg`를 저장한다. RV64는 그림 30처럼 짝수 번호 `pmpcfg0`, `pmpcfg2`, …, `pmpcfg14` 8개에 저장한다. RV64에서 홀수 번호 `pmpcfg1`, `pmpcfg3`, …, `pmpcfg15`는 불법이다.

> **주석** RV64에서 항목 8~15를 `pmpcfg1`이 아닌 `pmpcfg2`에 넣으면 RV32와 RV64 모두 항목 8~11이 `pmpcfg2[31:0]`에 있어 여러 MXLEN 지원 비용을 줄인다.

그림 29. RV32 PMP 설정 CSR 배치

CSR	31:24	23:16	15:8	7:0
pmpcfg0	pmp3cfg	pmp2cfg	pmp1cfg	pmp0cfg
pmpcfg1	pmp7cfg	pmp6cfg	pmp5cfg	pmp4cfg
⋮	⋮	⋮	⋮	⋮
pmpcfg15	pmp63cfg	pmp62cfg	pmp61cfg	pmp60cfg

그림 30. RV64 PMP 설정 CSR 배치 각 설정은 8비트

CSR	63:32 상위 설정 순서	31:0 하위 설정 순서
pmpcfg0	pmp7cfg pmp6cfg pmp5cfg pmp4cfg	pmp3cfg pmp2cfg pmp1cfg pmp0cfg
pmpcfg2	pmp15cfg pmp14cfg pmp13cfg pmp12cfg	pmp11cfg pmp10cfg pmp9cfg pmp8cfg
⋮	⋮	⋮
pmpcfg14	pmp63cfg pmp62cfg pmp61cfg pmp60cfg	pmp59cfg pmp58cfg pmp57cfg pmp56cfg

주소 레지스터는 `pmpaddr0~pmpaddr63`이다. RV32에서는 34비트 물리 주소의 비트 33:2를, RV64에서는 56비트 주소의 비트 55:2를 인코딩한다. 모든 물리 주소 비트를 구현할 필요는 없으므로 `pmpaddr`는 WARL이다.

> **주석** Sv32는 RV32에서 34비트 물리 주소를 지원하므로 PMP도 XLEN보다 넓은 주소를 지원해야 한다. Sv39 및 Sv48은 56비트 물리 주소 공간을 지원하므로 RV64 PMP도 같은 한도를 둔다.

그림 31. RV32 PMP 주소 레지스터 형식

비트 범위	필드
31:0	address[33:2] WARL

그림 32. RV64 PMP 주소 레지스터 형식

비트 범위	필드
63:54	0 WARL
53:0	address[55:2] WARL

그림 33의 R, W, X는 각각 읽기·쓰기·명령어 실행 허용을 나타낸다. 0이면 해당 접근을 거부한다. 세 필드는 합쳐 하나의 WARL 필드이며 R=0, W=1 조합은 예약이다. 나머지 A와 L은 다음 절에서 설명한다.

그림 33. PMP 설정 레지스터 형식

비트 범위	필드
7	L
6:5	0
4:3	A
2	X
1	W
0	R

실행 권한 없는 영역의 명령어 인출은 명령어 접근 폴트, 읽기 권한 없는 영역의 로드·LR·캐시 블록 관리 명령어는 로드 접근 폴트를 발생시킨다. 쓰기 권한 없는 영역의 스토어·SC·AMO·캐시 블록 0 채우기 명령어는 스토어 접근 폴트를 발생시킨다.

2.1.7.1.1. 주소 일치

설정 레지스터의 A는 대응 주소 레지스터의 주소 일치 모드를 표 13처럼 인코딩한다. A=0이면 비활성화되어 어떤 주소와도 일치하지 않는다. 다른 방식은 자연 정렬된 2의 거듭제곱 영역 NAPOT(4바이트 특수 경우 NA4 포함), 임의 범위의 상한 TOR이다. 이들은 4바이트 단위를 지원한다.

표 13. PMP 설정 레지스터 A 필드 인코딩

A	이름	설명
0	OFF	빈 영역 비활성화
1	TOR	범위의 상한
2	NA4	자연 정렬된 4바이트 영역
3	NAPOT	자연 정렬된 2의 거듭제곱 영역 8바이트 이상

NAPOT는 주소 레지스터의 하위 비트로 표 14처럼 크기를 인코딩한다.

표 14. PMP 주소 및 설정 레지스터의 NAPOT 범위 인코딩

pmpaddr	pmpcfg.A	일치 유형 및 크기
yyyy…yyyy	NA4	4바이트 NAPOT 영역
yyyy…yyy0	NAPOT	8바이트 NAPOT 영역
yyyy…yy01	NAPOT	16바이트 NAPOT 영역
yyyy…y011	NAPOT	32바이트 NAPOT 영역
…	…	…
yy01…1111	NAPOT	2^{XLEN}바이트 NAPOT 영역
y011…1111	NAPOT	2^{XLEN+1}바이트 NAPOT 영역
0111…1111	NAPOT	2^{XLEN+2}바이트 NAPOT 영역
1111…1111	NAPOT	2^{XLEN+3}바이트 NAPOT 영역

TOR이면 대응 주소 레지스터가 범위 상한, 바로 앞 주소 레지스터가 하한이다. 항목 i의 A=TOR이면 $\texttt{pmpcfg}_{i-1}$ 값과 관계없이 $\texttt{pmpaddr}_{i-1} \le y < \texttt{pmpaddr}_i$인 주소 y와 일치한다. 항목 0의 A=TOR이면 하한은 0이므로 $y < \texttt{pmpaddr}_0$와 일치한다.

> **주석** $\texttt{pmpaddr}_{i-1} \ge \texttt{pmpaddr}_i$이고 A=TOR이면 어떤 주소와도 일치하지 않는다.

> **주석** 2^{XLEN+2}를 상한으로 표현할 수 없으므로 34비트 물리 주소를 쓰는 RV32의 최상위 워드에 TOR로 낮은 모드의 접근을 허용할 수 없다. NAPOT를 사용하거나 낮은 모드에 접근을 허용하지 않으면 된다. 슈퍼바이저가 기본 페이지 단위로 관리하면 16GiB 공간의 0.00002%만 잃는다.

PMP는 4바이트까지 지원하지만 플랫폼은 더 큰 단위를 지정할 수 있다. 일반적으로 PMP 단위는 2^{G+2}바이트이며 모든 PMP 영역에서 같아야 한다. G≥1이면 NA4를 선택할 수 없다. G≥2이고 $\texttt{pmpcfg}_i\texttt{.A[1]}$=1, 즉 NAPOT이면 $\texttt{pmpaddr}_i\texttt{[G-2:0]}$은 모두 1로 읽힌다. G≥1이고 A[1]=0, 즉 OFF/TOR이면 $\texttt{pmpaddr}_i\texttt{[G-1:0]}$은 모두 0으로 읽힌다. 이 하위 비트는 TOR 주소 일치 논리에 영향을 주지 않는다.

A[1] 변경은 pmpaddr의 읽기 값에 영향을 주지만 실제 저장 값에는 영향을 주지 않는다. 특히 A를 NAPOT→TOR/OFF→NAPOT로 바꾸면 $\texttt{pmpaddr}_i\texttt{[G-1]}$은 원래 값을 유지한다.

주석 pmp0cfg에 0, pmpaddr0에 모든 비트가 1인 값을 쓴 뒤 pmpaddr0을 읽어 PMP 단위를 알아낼 수 있다. 읽은 값에서 가장 낮은 1비트의 인덱스가 G이면 단위는 2^{G+2}바이트이다.

2.1.7.1.2. 잠금과 특권 모드

L은 PMP 항목의 잠금을 나타낸다. 잠기면 설정 및 관련 주소 레지스터 쓰기를 무시하며 하트 리셋까지 유지된다. 항목 i가 잠기면 $\texttt{pmp}_i\texttt{cfg}$와 $\texttt{pmpaddr}_i$ 쓰기를 무시한다. 추가로 A=TOR이면 $\texttt{pmpaddr}_{i-1}$ 쓰기도 무시한다.

주석 A=OFF여도 L을 설정하면 잠긴다.

L은 잠금 외에 R/W/X를 M모드에도 적용할지 나타낸다. L=1이면 모든 특권 모드에 적용한다. L=0이면 해당 항목과 일치하는 M모드 접근은 항상 성공하고 R/W/X는 S/U에만 적용한다.

2.1.7.1.3. 우선순위 및 일치 논리

일부 구현에서는 RVWMO 명세와 같이 비정렬 로드·스토어·명령어 인출을 여러 메모리 연산으로 분해하고, 접근 폴트 전에 일부가 성공할 수 있다. 각 연산을 독립적으로 PMP 검사하므로 비정렬 스토어 일부가 실패해도 통과한 다른 부분은 이미 보일 수 있다. RV32D의 FSD처럼 XLEN보다 넓은 스토어도 자연 정렬 여부와 관계없이 같은 현상이 가능하다.

PMP 항목의 우선순위는 정적이다. 메모리 연산의 어느 바이트와든 일치하는 항목 중 번호가 가장 작은 항목이 성공·실패를 결정한다. 이 항목은 연산의 모든 바이트와 일치해야 하며, 그렇지 않으면 L/R/W/X와 관계없이 실패한다. 예를 들어 주소 0xC~0xF와 일치하는 항목이 최고 우선순위라면 0x8~0xF 8바이트 접근은 실패한다.

모든 바이트와 일치하면 L/R/W/X로 판단한다. L=0이고 접근 모드가 M이면 성공한다. 그렇지 않으면, 즉 L=1이거나 S/U 접근이면 해당 접근 유형의 R/W/X가 1이어야 성공한다. 어떤 항목도 일치하지 않는 M모드 연산은 성공한다. S/U 연산에 일치 항목이 없고 PMP 항목이 하나 이상 구현되어 있으면 실패한다.

주석 PMP 항목이 하나 이상 있지만 모든 A=OFF이면 S/U의 모든 메모리 접근이 실패한다.

실패한 연산은 명령어·로드·스토어 접근 폴트를 발생시킨다. 단일 명령어가 여러 메모리 연산을 생성할 수 있고 상호 원자적이지 않을 수 있다. 하나라도 실패하면 접근 폴트가 발생하지만 다른 연산은 성공하여 눈에 보이는 부수 효과를 남길 수 있다. 특히 가상 메모리 참조 명령어는 여러 메모리 연산으로 분해된다.

2.1.7.2. 물리 메모리 보호와 페이징

PMP는「슈퍼바이저」에 설명한 페이지 기반 가상 메모리와 함께 동작하도록 설계되었다. 페이징을 켜면 가상 메모리 명령어가 페이지 테이블의 암묵적 참조를 포함하여 여러 물리 접근을 만들 수 있으며 모두 PMP를 적용한다. 암묵적 페이지 테이블 접근의 유효 모드는 S이다.

가상 메모리 구현은 명시적 접근에 필요하기 전에 추측적으로 주소를 변환하고 주소 변환 캐시에 저장할 수 있다. Bare 변환 및 M모드의 유효 주소=물리 주소 동일 매핑도 캐시할 수 있다. 결과 물리 주소의 PMP 설정은 주소 변환부터 명시적 접근 사이 어느 시점에든 검사하거나 캐시할 수 있다. 따라서 PMP 변경 후 M모드 소프트웨어는 가상 메모리 시스템 및 모든 PMP·주소 변환 캐시와 동기화해야 한다. PMP CSR에 쓴 다음 rs1=x0, rs2=x0인 SFENCE.VMA를 실행하면 된다. H 확장의 추가 요구사항은「슈퍼바이저 메모리 관리 펜스 명령어」를 참조한다.

페이지 기반 가상 메모리를 구현하지 않으면 접근 시 동기적으로 PMP를 검사하므로 SFENCE.VMA가 필요하지 않다.

출처 및 이용 조건

원저작물 RISC-V International 및 원저자. The RISC-V Instruction Set Manual. 공식 릴리스 20260120. 이 문서는 요청된 웹페이지 본문의 한국어 번역이며 RISC-V International의 공식 번역이 아니다. 웹사이트 탐색 메뉴와 공통 푸터는 제외했다.

© RISC-V International. 원저작물은 Creative Commons Attribution 4.0 International CC BY 4.0으로 제공된다. 한국어 번역과 서식 재구성을 적용했다. 원저작물의 보증 부인 및 라이선스 조건은 아래 링크를 따른다.

원 특권 명세 1.9.1 저작권 © 2010–2017 Andrew Waterman, Yunsup Lee, Rimas Avižienis, David Patterson, Krste Asanović. CC BY 4.0.

CC BY 4.0 라이선스

공식 원문 및 기여자